

人工智能学院2024级本科生课程

《机器学习》

第九章 聚类

吉林大学 人工智能学院

授课教师：王鑫

内 容 大 纲

- 无监督学习
- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

无监督学习

监督学习

所有样本都带有标签

$Y=F(X)$, Y 是监督信号, 机器使用已知样本及标签预测新的标签 (已知标签)

回归

估计连续变量
输出: 实数值

分类

预测离散变量 (类别)
输出: 离散值、布尔量、定类变量

无监督学习

样本没有标签, 需要学习和发现样本中的内在性质与规律

人工输入未定义结果的数据, 机器发现隐藏在数据中的有用信息。

聚类

对样本进行分组

密度估计

近似分布

降维

选择相关特征或变量

内 容 大 纲

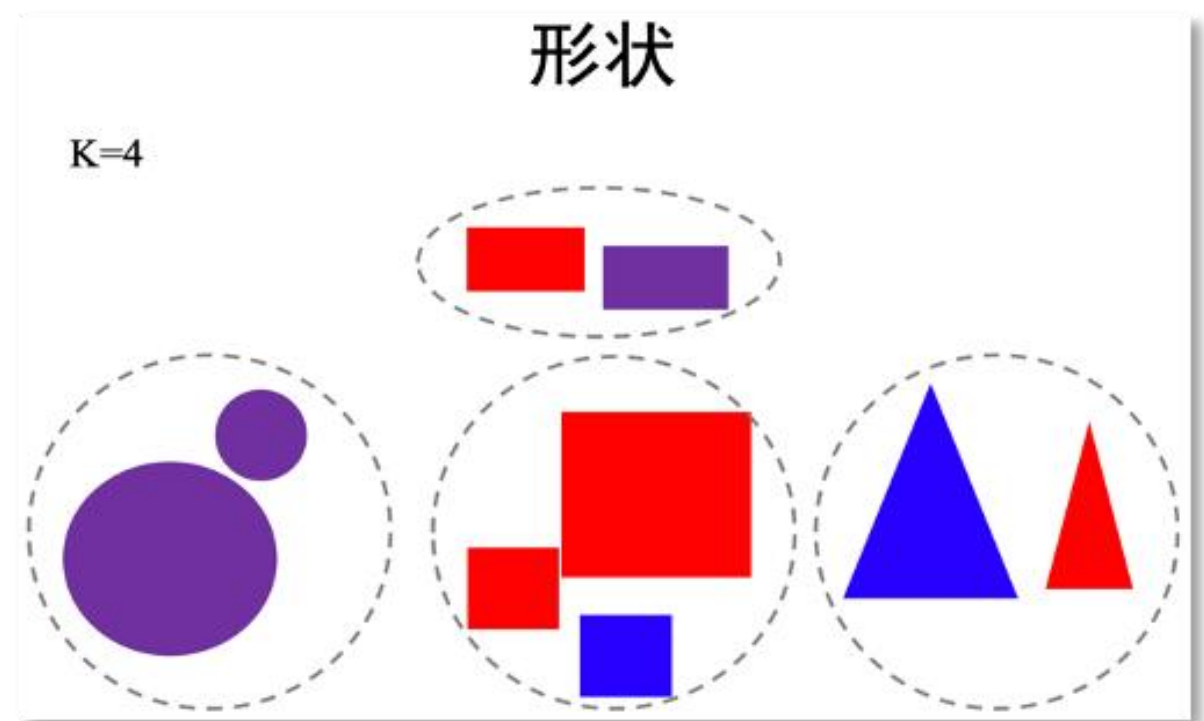
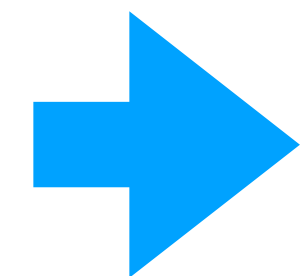
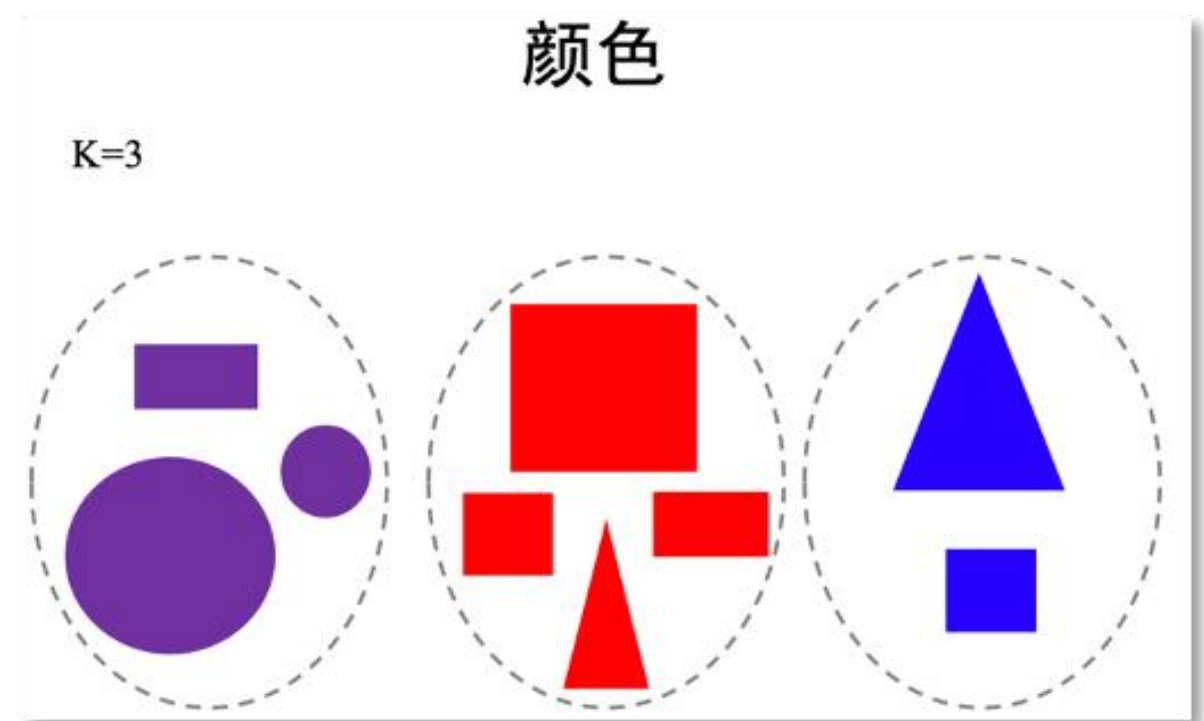
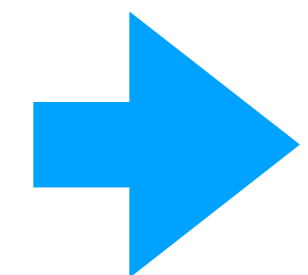
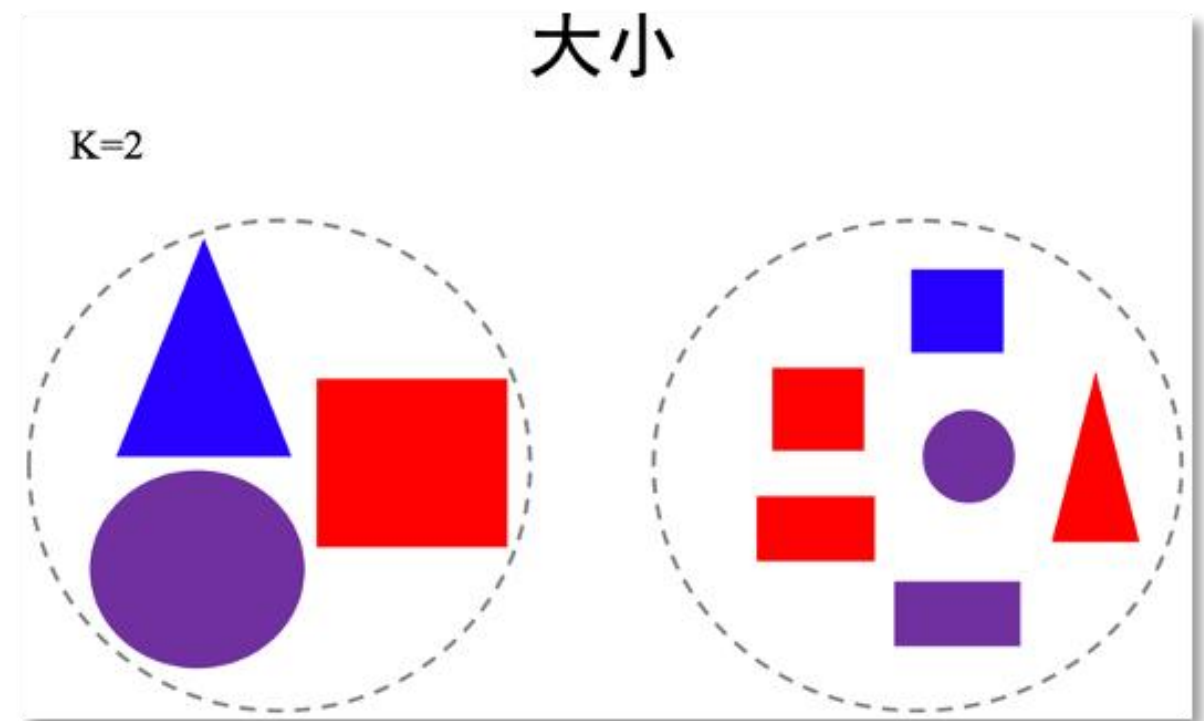
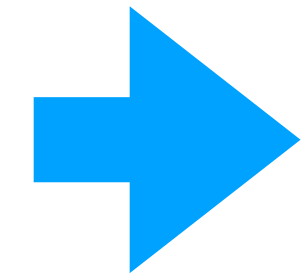
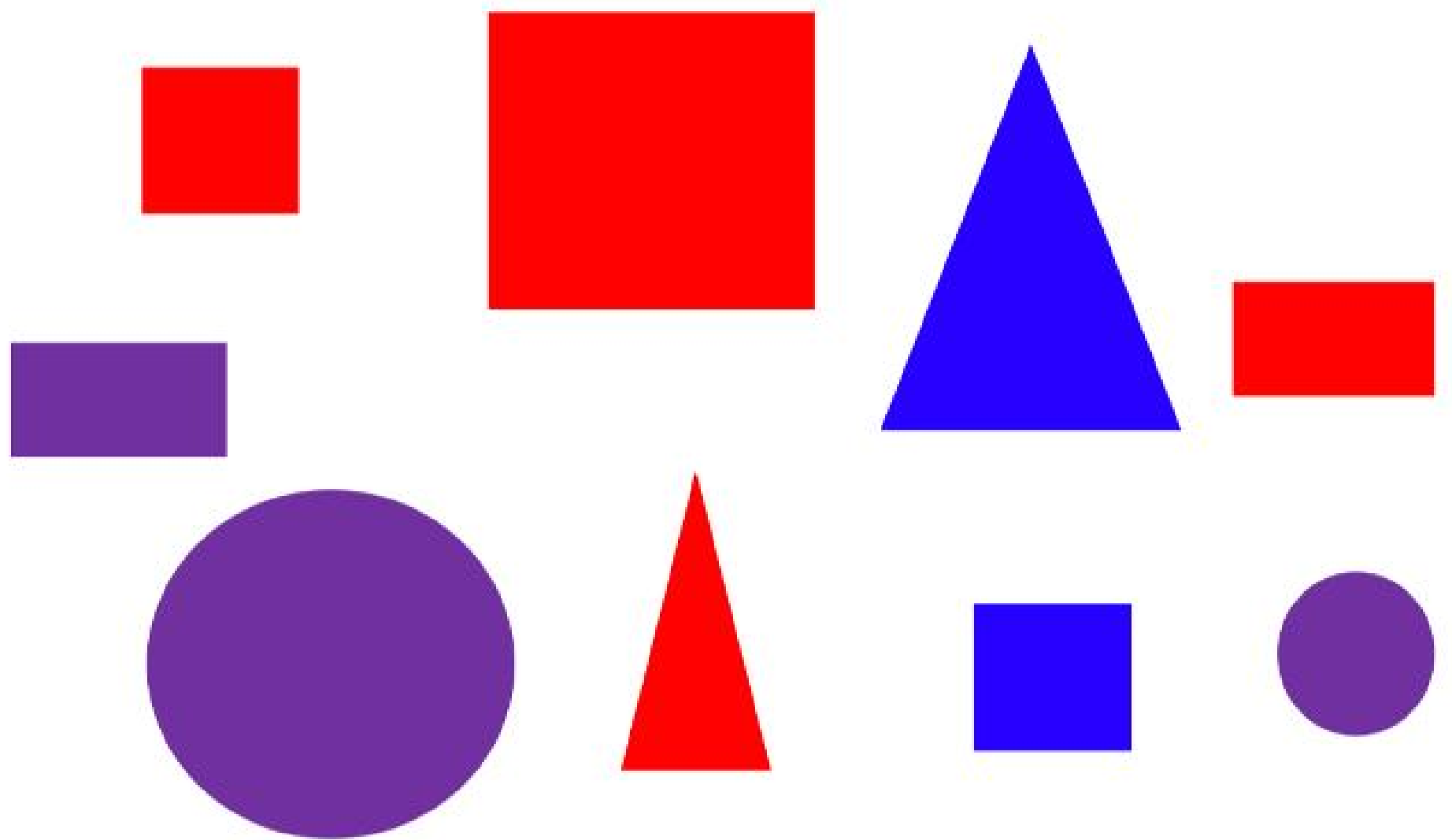
- 无监督学习
- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

聚类任务

“物以类聚”

聚类

✓对下面图形进行分类(幼儿园作业)



聚类任务 - 典型“无监督学习”任务之一

- 无监督学习目标
 - 通过对无标记训练样本的学习，揭示数据的内在性质及规律，为进一步的数据分析提供基础。
- 聚类
 - 研究最多、应用最广泛的无监督学习任务
 - 将数据样本划分为若干个不相交的子集，每个子集称为一个“簇”(cluster)。
 - “簇”对应一个潜在的概念（类别），这些概念事先不知道，概念语义可能由用户自己命名。

聚类任务 - 典型“无监督学习”任务

- 形式化描述

- 样本集合 $D = \{x_1, x_2, \dots, x_m\}$ 包含 m 个无标记样本，每个样本 $x_i = (x_{i1}; x_{i2}; \dots; x_{in})$ 是一个 n 维特征向量，则聚类算法将样本集合 D 划分为 k 个不相交的簇 $\{C_l | l = 1, 2, \dots, k\}$ ，其中不同簇交集为 ϕ 且并集为 D 。 $\lambda_j \in \{1, 2, \dots, k\}$ 表示样本 x_j 的“簇标签”。聚类的结果是 m 个样本的簇标记向量 $\lambda = (\lambda_1; \lambda_2; \dots; \lambda_m)$ 。

- 聚类任务扩展说法：

- 单独过程，寻找数据内在的分布结构
- 作为分类等其他学习任务的前驱过程，作为特征提取、类别预测的重要支撑。

内 容 大 纲

- 无监督学习
- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

性能度量

- 什么是好的聚类?
 - 有效性指标 (Validity Index)
- 性能度量目标
 - 评估聚类结果的**好坏**
 - 确立**优化目标**，得到更好的聚类结果
- 基本想法 (“物以类聚”)
 - “簇内相似度” (intra-cluster similarity) 高：簇内样本相似
 - “簇间相似度” (inter-cluster similarity) 低：簇间样本尽可能不同

性能度量

- 外部指标

- 将聚类结果与某个“参考模型”(reference model) 进行比较

- 形式化描述

- 样本集合 $D = \{x_1, x_2, \dots, x_m\}$, 聚类的簇划分为 $C = \{C_1, C_2, \dots, C_k\}$, 参考模型的簇划分为 $C^* = \{C_1^*, C_2^*, \dots, C_s^*\}$, $k \neq s$, 令 λ 和 λ^* 表示 C 和 C^* 对应的簇标记向量。我们将样本两两配对, 定义基本指标如下:

- $a = |SS|, SS = \{(x_i, x_j) | \lambda_i = \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}$

- $b = |SD|, SD = \{(x_i, x_j) | \lambda_i = \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}$

- $c = |DS|, DS = \{(x_i, x_j) | \lambda_i \neq \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}$

- $d = |DD|, DD = \{(x_i, x_j) | \lambda_i \neq \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}$

性能度量

- 外部指标（举例）

聚类结果 C			参考结果C*			
C1: x1, x2, x3			C1*: x1, x2, x4			
C2: x4, x5			C2*: x3, x5			
x1	x2	x3	x4	x5	合计	
a= SS : (x1,x2)						1
b= SD : (x1,x3)		(x2,x3)		(x4,x5)		3
c= DS : (x1,x4)	(x2,x4)	(x3,x5)				2
d= DD : (x1,x5)	(x2,x5)	(x3,x4)				4

性能度量

- 常用聚类性能度量外部指标

- Jaccard 系数 (Jaccard Coefficient, 简称 JC)

- $$JC = \frac{a}{a+b+c}$$

- FM 指数 (Fowlkes and Mallows Index, 简称 FMI)

- $$FMI = \sqrt{\frac{a}{a+b} \cdot \frac{a}{a+c}}$$

- Rand 指数 (Rand Index, 简称 RI)

- $$RI = \frac{2(a+d)}{m(m-1)}$$

- 性能度量的结果值均在 $[0, 1]$ 区间, 值越大越好.

性能度量

- 内部指标
 - 直接考察聚类结果而不利用任何参考模型
- 给定簇划分 $C = \{C_1, C_2, \dots, C_k\}$, 常用距离定义如下

- 平均距离:
$$\text{avg}(C) = \frac{2}{|C|(|C|-1)} \sum_{1 \leq i < j \leq |C|} \text{dist}(\mathbf{x}_i, \mathbf{x}_j)$$

- 簇内最大距离:
$$\text{diam}(C) = \max_{1 \leq i < j \leq |C|} \text{dist}(\mathbf{x}_i, \mathbf{x}_j)$$

- 簇间最小距离:
$$d_{\min}(C_i, C_j) = \min_{\mathbf{x}_i \in C_i, \mathbf{x}_j \in C_j} \text{dist}(\mathbf{x}_i, \mathbf{x}_j)$$

- 两个簇中心点间的距离:
$$d_{\text{cen}}(C_i, C_j) = \text{dist}(\mu_i, \mu_j)$$

前两者都是簇内距离, 越小越好; 后两者都是簇间距离, 越大越好

性能度量*

- 常用聚类性能度量内部指标

- DB 指数 (Davies- Bouldin Index, 简称 DBI)

- $$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{\text{avg}(C_i) + \text{avg}(C_j)}{d_{\text{cen}}(\mu_i, \mu_j)} \right)$$
 $\frac{\text{簇内平均距离} \downarrow}{\text{簇中心距离} \uparrow}$

- Dunn指数(Dunn Index,简称 DI)

- $$DI = \min_{1 \leq i \leq k} \left\{ \min_{j \neq i} \left(\frac{d_{\min}(C_i, C_j)}{\max_{1 \leq l \leq k} \text{diam}(C_l)} \right) \right\}$$
 $\frac{\text{簇间样本最小距离} \uparrow}{\text{簇内样本最大距离} \downarrow}$

DBI 的值越小越好，而 DI 则相反，值越大越好。

内 容 大 纲

- 无监督学习
- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

距离计算

- “距离度量” (Distance Measure) 满足基本性质：
 - 非负性： $\text{dist}(\mathbf{x}_i, \mathbf{x}_j) \geq 0$
 - 同一性： $\text{dist}(\mathbf{x}_i, \mathbf{x}_j) = 0$ 当且仅当 $\mathbf{x}_i = \mathbf{x}_j$
 - 对称性： $\text{dist}(\mathbf{x}_i, \mathbf{x}_j) = \text{dist}(\mathbf{x}_j, \mathbf{x}_i)$
 - 直递性： $\text{dist}(\mathbf{x}_i, \mathbf{x}_j) \leq \text{dist}(\mathbf{x}_i, \mathbf{x}_k) + \text{dist}(\mathbf{x}_k, \mathbf{x}_j)$ “三角不等式”
- “闵可夫斯基距离”(Minkowski distance)
 - 给定样本 $\mathbf{x}_i = (x_{i1}; x_{i2}; \cdots; x_{in})$ 与 $\mathbf{x}_j = (x_{j1}; x_{j2}; \cdots; x_{jn})$
 - $\text{dist}_{\text{mk}}(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{u=1}^n |x_{iu} - x_{ju}|^p \right)^{\frac{1}{p}}$, p 大于等于1时, 满足上面基本性质。

距离计算

- “闵可夫斯基距离”几种特例：

- $p=2$ ，欧氏距离 (Euclidean distance)

- $\text{dist}_{\text{ed}}(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2 = \sqrt{\sum_{u=1}^n |x_{i u} - x_{j u}|^2}$

- $p=1$ ，曼哈顿距离 (Manhattan distance)

- $\text{dist}_{\text{man}}(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_1 = \sum_{u=1}^n |x_{i u} - x_{j u}|$

- 数据属性划分

- 连续属性(continuous attribute)和离散属性(categorical attribute)

- 有序属性(ordinal attribute)和无序属性 (non-ordinal attribute)

在讨论距离计算时，特别是离散属性，属性上是否定义了“序”关系更为重要。

距离计算

- 无序属性可采用 VDM (Value Difference Metric)

$$\text{VDM}_p(a, b) = \sum_{i=1}^k \left| \frac{m_{u,a,i}}{m_{u,a}} - \frac{m_{u,b,i}}{m_{u,b}} \right|^p$$

- 将闵可夫斯基距离和VDM 结合即可处理混合属性

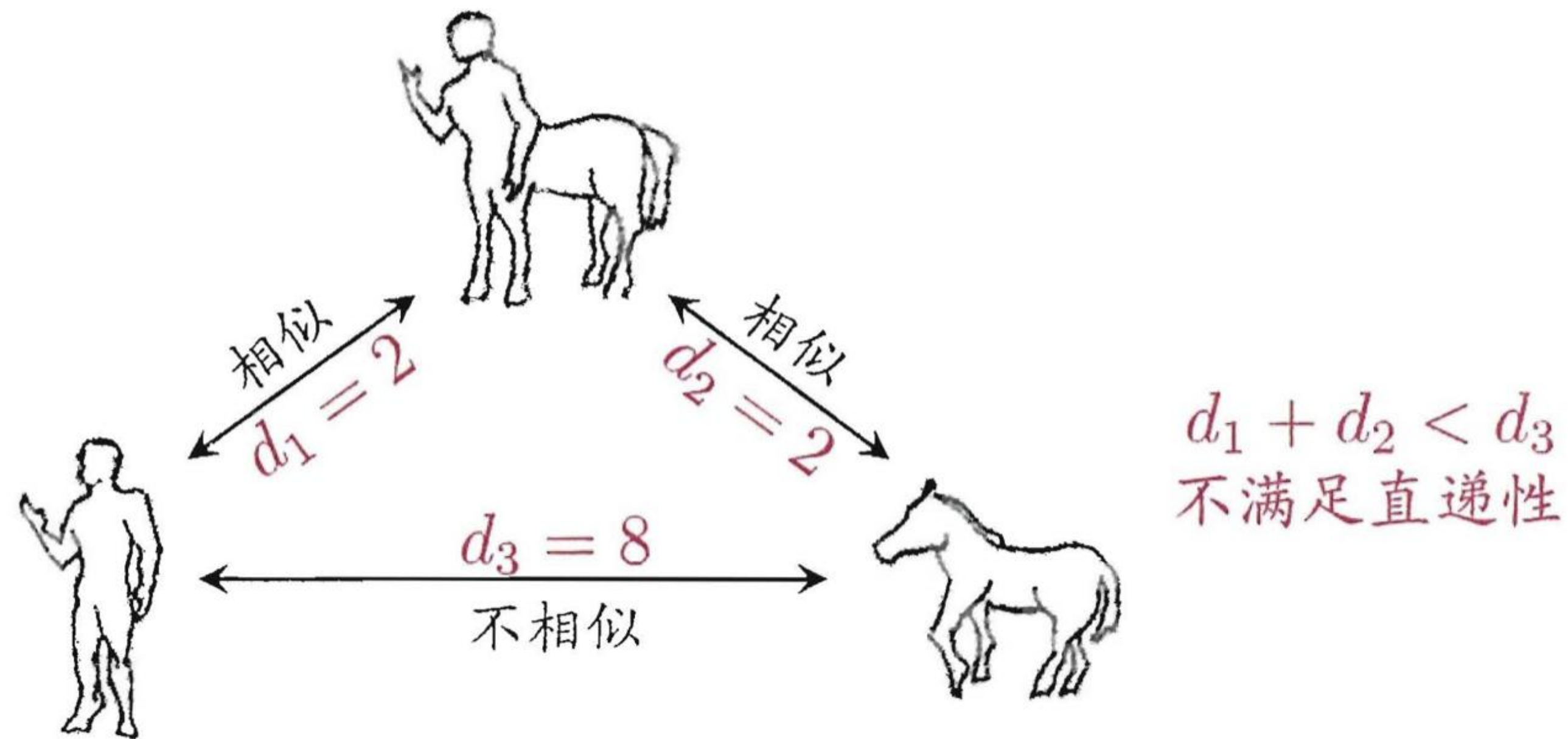
$$\text{MinkovDM}_p(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{u=1}^{n_c} |x_{i,u} - x_{j,u}|^p + \sum_{u=n_c+1}^n \text{VDM}_p(x_{i,u}, x_{j,u}) \right)^{\frac{1}{p}}$$

- 加权闵可夫斯基距离

$$\text{dist}_{\text{wmk}}(\mathbf{x}_i, \mathbf{x}_j) = \left(w_1 \cdot |x_{i,1} - x_{j,1}|^p + \cdots + w_n \cdot |x_{i,n} - x_{j,n}|^p \right)^{\frac{1}{p}}$$

距离计算

- 相似性度量 (Similarity Measure)
 - 距离越大，相似度越小
 - 相反，相似性度量的距离未必满足度量的所有基本性质
- “非度量距离”(non-metric distance)



距离计算

- 计算距离实现（以欧式距离为例）
 - Python

```
import math
# 计算两点之间的距离
1 usage
def eucliDist(A,B):
    return math.sqrt(sum([(a - b)**2 for (a,b) in zip(A,B)]))
X = [1,2,3,4]
Y = [0,1,2,3]
print(eucliDist(X,Y))
```

- Numpy

```
import numpy

dist = numpy.sqrt(numpy.sum(numpy.square(vec1 - vec2)))

dist = numpy.linalg.norm(vec1 - vec2)
```

内 容 大 纲

- 无监督学习
- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

原型聚类

- “基于原型的聚类”(Prototype-based Clustering)
 - 此类算法假设聚类结构能通过一组原型(样本空间中具有代表性的点)刻画，在现实聚类任务中极为常用。
 - 通常情形下，算法先对原型进行初始化，然后对原型进行迭代更新求解。采用不同的原型表示、不同的求解方式，将产生不同的算法。
- 代表方法：
 - K均值聚类，学习向量量化(LVQ)，高斯混合聚类

原型聚类

- “基于原型的聚类”(Prototype-based Clustering)
 - K均值聚类：随机选择K个样本作为原型，将簇的均值向量作为原型表示；
 - 学习向量量化(LVQ)：由于每个样本是有标签，将标签作为辅助，随机选择K个标签作为原型，初始化在相应标签的样本中随机选择样本作为原型向量；
 - 高斯混合聚类：采用概率模型表达聚类原型，选择K个高斯混合成分。

原型聚类-K均值

- K-均值聚类

- 给定样本集 $D = \{x_1, x_2, \dots, x_m\}$, “K均值”算法针对聚类所获得簇划分 $C = \{C_1, C_2, \dots, C_k\}$ 最小化平方误差

$$\bullet E = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|_2^2$$

- 其中 $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$ 是簇 C_i 的均值向量, 刻画了簇内样本围绕簇均值向量的紧密程度, E 值越小, 则簇内样本相似度越高

- 最小化上述公式是NP难的问题, K均值算法采用了贪心策略, 通过迭代优化近似求解上述公式

原型聚类-K均值

K 均 值 聚 类 算 法 流 程

输入: 样本集 $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$;

聚类簇数 k .

过程:

1: 从 D 中随机选择 k 个样本作为初始均值向量 $\{\mu_1, \mu_2, \dots, \mu_k\}$

2: **repeat**

3: 令 $C_i = \emptyset$ ($1 \leq i \leq k$)

4: **for** $j = 1, 2, \dots, m$ **do**

5: 计算样本 $\mathbf{x}_j$ 与各均值向量 μ_i ($1 \leq i \leq k$) 的距离: $d_{ji} = \|\mathbf{x}_j - \mu_i\|_2$;

6: 根据距离最近的均值向量确定 $\mathbf{x}_j$ 的簇标记: $\lambda_j = \arg \min_{i \in \{1, 2, \dots, k\}} d_{ji}$;

7: 将样本 $\mathbf{x}_j$ 划入相应的簇: $C_{\lambda_j} = C_{\lambda_j} \cup \{\mathbf{x}_j\}$;

8: **end for**

9: **for** $i = 1, 2, \dots, k$ **do**

10: 计算新均值向量: $\mu'_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$;

11: **if** $\mu'_i \neq \mu_i$ **then**

12: 将当前均值向量 μ_i 更新为 μ'_i

13: **else**

14: 保持当前均值向量不变

15: **end if**

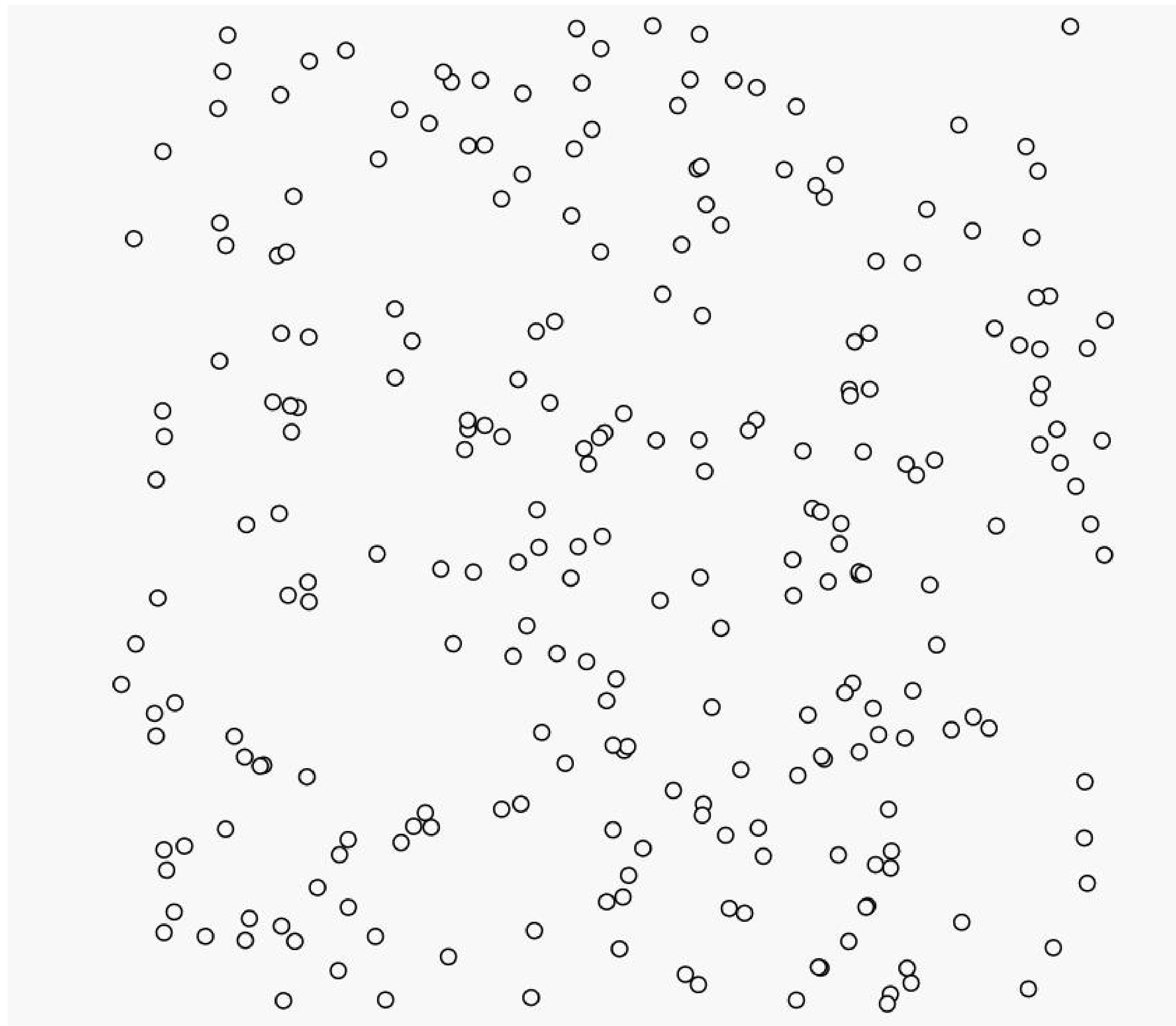
16: **end for**

17: **until** 当前均值向量均未更新

输出: 簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

原型聚类-K均值

K
均
值
聚
类
举
例



原型聚类-K均值

K
均
值
聚
类
举
例

编号	密度	含糖率	编号	密度	含糖率	编号	密度	含糖率
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459

原型聚类-K均值

假定聚类簇数 $k = 3$, 算法开始时随机选取三个样本 $\mathbf{x}_6, \mathbf{x}_{12}, \mathbf{x}_{27}$ 作为初始均值向量, 即

$$\mu_1 = (0.403; 0.237), \mu_2 = (0.343; 0.099), \mu_3 = (0.532; 0.472).$$

考察样本 $\mathbf{x}_1 = (0.697; 0.460)$, 它与当前均值向量 μ_1, μ_2, μ_3 的距离分别为 0.369, 0.506, 0.166, 因此 $\mathbf{x}_1$ 将被划入簇 C_3 中. 类似的, 对数据集中的所有样本考察一遍后, 可得当前簇划分为

$$C_1 = \{\mathbf{x}_5, \mathbf{x}_6, \mathbf{x}_7, \mathbf{x}_8, \mathbf{x}_9, \mathbf{x}_{10}, \mathbf{x}_{13}, \mathbf{x}_{14}, \mathbf{x}_{15}, \mathbf{x}_{17}, \mathbf{x}_{18}, \mathbf{x}_{19}, \mathbf{x}_{20}, \mathbf{x}_{23}\};$$

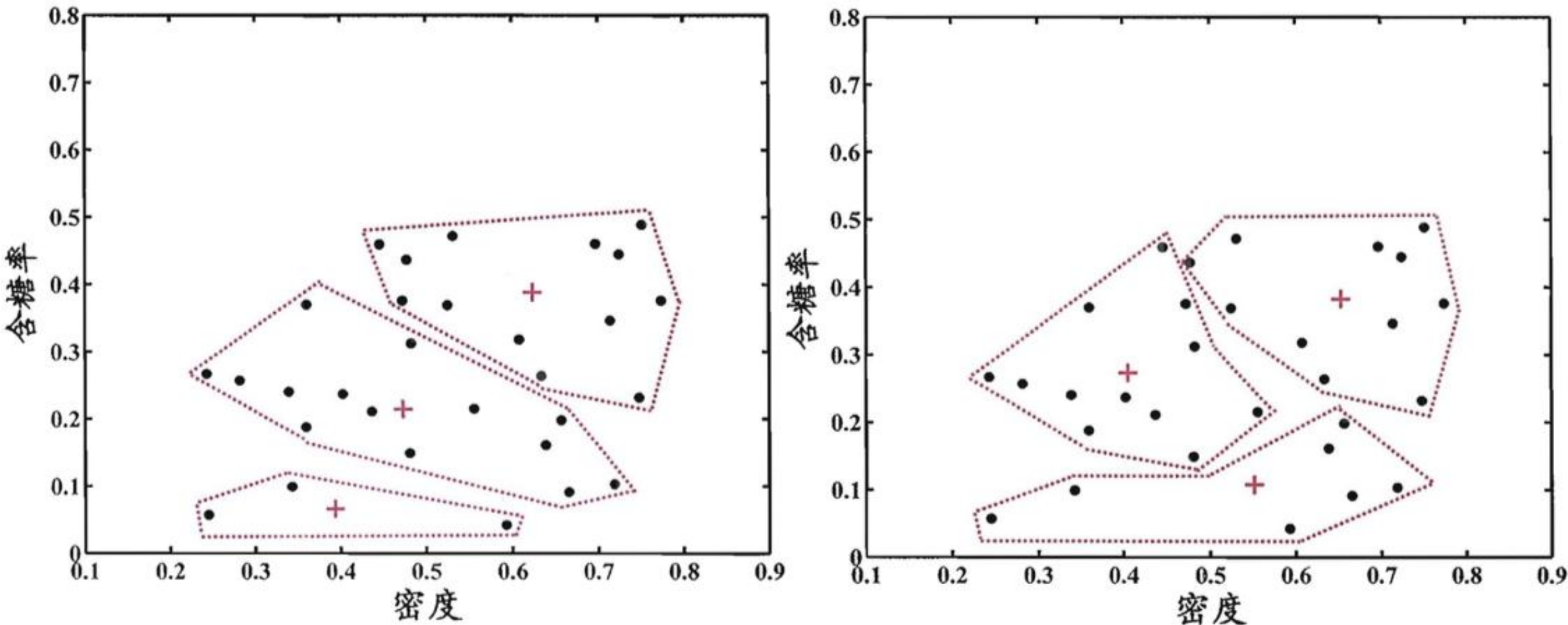
$$C_2 = \{\mathbf{x}_{11}, \mathbf{x}_{12}, \mathbf{x}_{16}\};$$

$$C_3 = \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4, \mathbf{x}_{21}, \mathbf{x}_{22}, \mathbf{x}_{24}, \mathbf{x}_{25}, \mathbf{x}_{26}, \mathbf{x}_{27}, \mathbf{x}_{28}, \mathbf{x}_{29}, \mathbf{x}_{30}\}.$$

于是, 可从 C_1, C_2, C_3 分别求出新的均值向量

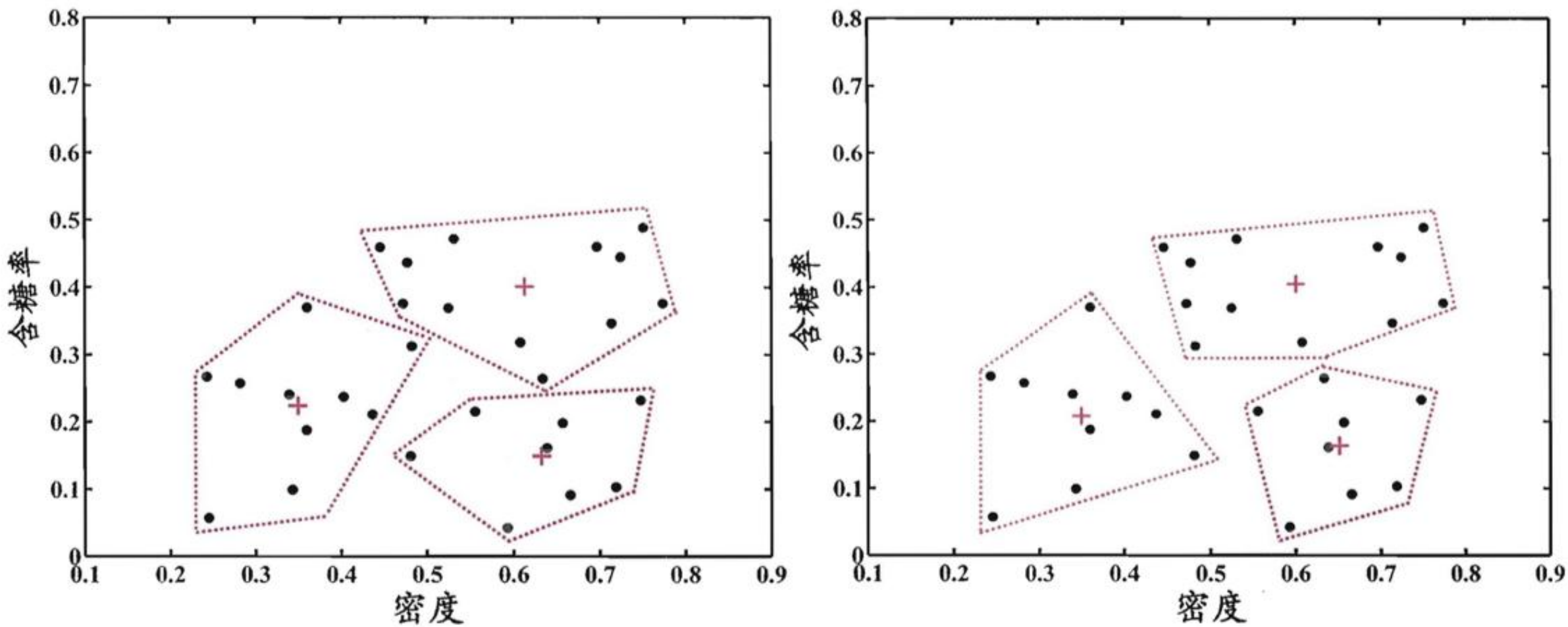
$$\mu'_1 = (0.473; 0.214), \mu'_2 = (0.394; 0.066), \mu'_3 = (0.623; 0.388).$$

更新当前均值向量后, 不断重复上述过程, 如图 9.3 所示, 第五轮迭代产生的结果与第四轮迭代相同, 于是算法停止, 得到最终的簇划分.



(a) 第一轮迭代后

(b) 第二轮迭代后



(c) 第三轮迭代后

(d) 第四轮迭代后

原型聚类-K均值优缺点

- 优点
 - 原理很简单，实现容易，算法收敛速度也很快
 - 聚类效果优，可解释性强。当收敛之后，清晰看到聚类的效果
 - 约束条件少。算法中只有参数K，对K调节得到最好的聚类效果
- 缺点
 - K值的估计是非常困难的，有时候通过交叉验证来获取
 - 迭代的方法得到的结果是局部最优解，而不能得到全局最优解
 - 对噪音和异常点很敏感，异常点对质心的确定影响很大

原型聚类-K均值（初始化优化）

- 初始化均值向量选择，对聚类结果和运行时间影响很大
- K-means++ 优化策略：
 - 随机选择一个点作为第一个聚类均值向量 μ_1
 - 对于数据集中的每一个点 x_i ，计算它与已选择的均值向量中最近聚类中心的距离 $D(x_i) = \arg\min ||x_i - \mu_r||^2$
 - 选择一个新的数据点作为新的均值向量，选择的原则是：
 $D(x)$ 较大的点，被选取作为聚类均值向量的概率较大
 - 重复以上两个步骤，直到选择出 k 个聚类均值向量
 - 利用这 k 个质心来作为初始化均值去运行K-Means算法

原型聚类-K均值（计算距离优化）

- 计算所有样本到所有质心的距离，这样会比较的耗时
- elkan K-Means利用两边之和大于等于第三边，以及两边之差小于第三边的三角形性质，来减少距离的计算
- 对于一个样本点 x 和两个均值向量 μ_{j_1} 和 μ_{j_2}
- 依据规律1：计算两个均值距离 $D(j_1, j_2)$ ，如果计算发现 $2D(x, j_1) \leq D(j_1, j_2)$ ，知道 $D(x, j_1) \leq D(x, j_2)$ ，不需要再计算 $D(x, j_2)$ ；
- 依据规律2：直接得到 $D(x, j_2) \geq \max\{0, D(x, j_1) - D(j_1, j_2)\}$

原型聚类-K均值（大样本优化）

- 如果样本量非常大，比如达到10万以上，特征有100以上，此时用传统的K-Means算法非常的耗时
- Mini Batch K-Means算法：
 - 选择一个合适的批样本大小batch size，仅仅用batch size个样本来做K-Means聚类
 - 无放回的随机采样得到批样本
 - 聚类的精确度会有一些降低，为了增加算法的准确性，一般会多跑几次Mini Batch K-Means算法

原型聚类-K均值与KNN区别和联系

KNN	K-Means
1. KNN是分类算法 2. 属于监督学习 3. 训练数据集是带label的数	1. K-Means是聚类算法 2. 属于非监督学习 3. 训练数据集是无labe的数据，是杂乱无章，经过聚类后变得有序，先无序，后有序。
没有明显的前期训练过程，属于memory based learning	有明显的前期训练过
K的含义：一个样本x，对它进行分类，就从训练数据集中，在x附近找离它最近的K个数据点，这K个数据点，类别c占的个数最多，就把x的label设为c。	K的含义：K是人工固定好的数字，假设数据集可以分为K个簇，那么就训练出这K个分类。

原型聚类- 学习向量量化

- “学习向量量化”(Learning Vector Quantization)
 - 寻找一组原型向量来刻画聚类结构，LVQ 假设数据样本带有类别标记，学习过程利用监督信息来辅助聚类。
- 形式化描述
 - 给定样本集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$ ，每个样本 $\mathbf{x}_j$ 的是由 n 个属性描述的特征向量 $(x_{j1}; x_{j2}; \dots; x_{jn})$ ， $y_j \in \mathcal{Y}$ 是样本 $\mathbf{x}_j$ 的类别标记。
 - LVQ 的目标是学得一组 n 维原型向量 $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_q\}$ ，每个原型向量代表一个聚类簇，簇标记 $t_i \in \mathcal{Y}$ 。

原型聚类- 学习向量化

- 算法描述 输入: 样本集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
原型向量个数 q , 各原型向量预设的类别标记 $\{t_1, t_2, \dots, t_q\}$;
学习率 $\eta \in (0, 1)$.

过程:

- 1: 初始化一组原型向量 $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_q\}$
- 2: **repeat**
- 3: 从样本集 D 随机选取样本 $(\mathbf{x}_j, y_j)$;
- 4: 计算样本 $\mathbf{x}_j$ 与 $\mathbf{p}_i$ ($1 \leq i \leq q$) 的距离: $d_{ji} = \|\mathbf{x}_j - \mathbf{p}_i\|_2$;
- 5: 找出与 $\mathbf{x}_j$ 距离最近的原型向量 $\mathbf{p}_{i^*}$, $i^* = \arg \min_{i \in \{1, 2, \dots, q\}} d_{ji}$;
- 6: **if** $y_j = t_{i^*}$ **then**
- 7: $\mathbf{p}' = \mathbf{p}_{i^*} + \eta \cdot (\mathbf{x}_j - \mathbf{p}_{i^*})$
- 8: **else**
- 9: $\mathbf{p}' = \mathbf{p}_{i^*} - \eta \cdot (\mathbf{x}_j - \mathbf{p}_{i^*})$
- 10: **end if**
- 11: 将原型向量 $\mathbf{p}_{i^*}$ 更新为 $\mathbf{p}'$
- 12: **until** 满足停止条件

输出: 原型向量 $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_q\}$

原型向量更新原则:

原型向量标记与类别标记相同, 原型向量向 $\mathbf{x}_j$ 样本靠拢;
相反, 原型向量远离样本 $\mathbf{x}_j$ 。

原型聚类- 学习向量化

- LVQ如何更新原型向量

- 若原型向量 p_i^* 与 x_j 的类别标记相同：令 p_i^* 向 x_j 的方向靠拢，如第7行所示，新原型向量为

- $$p' = p_{i^*} + \eta \cdot (x_j - p_{i^*})$$

- p' 与 x_j 之间的距离为

- $$\begin{aligned} \|p' - x_j\|_2 &= \|p_{i^*} + \eta \cdot (x_j - p_{i^*}) - x_j\|_2 \\ &= (1 - \eta) \cdot \|p_{i^*} - x_j\|_2 \end{aligned}$$

- 令学习率 $\eta \in (0,1)$ ，则原型向量 p_i^* 在更新为 p' 之后将更接近 x_j

原型聚类- 学习向量化

- LVQ如何更新原型向量
 - 若原型向量 $\mathbf{p}_i^*$ 与 $\mathbf{x}_j$ 的类别标记不同：更新后的原型向量与 $\mathbf{x}_j$ 之间的距离将增大为 $(1 + \eta) \cdot \|\mathbf{p}_{i^*} - \mathbf{x}_j\|_2$ ，从而更远离 $\mathbf{x}_j$
- LVQ如何产生簇划分
 - 获得原型向量 $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_q\}$ 后，给定任意样本 $\mathbf{x}$ ，将其划入与其距离最近的原型向量所代表的簇中，即每个原型向量 $\mathbf{p}_i$ 定义了与之相关的一个区域 R_i ，该区域中每个样本与 $\mathbf{p}_i$ 的距离不大于它与其他原型向量 $\mathbf{p}_{i'} (i' \neq i)$ 的距离，即

$$R_i = \{\mathbf{x} \in \mathcal{X} \mid \|\mathbf{x} - \mathbf{p}_i\|_2 \leq \|\mathbf{x} - \mathbf{p}_{i'}\|_2, i' \neq i\}$$

原型聚类-学习向量化

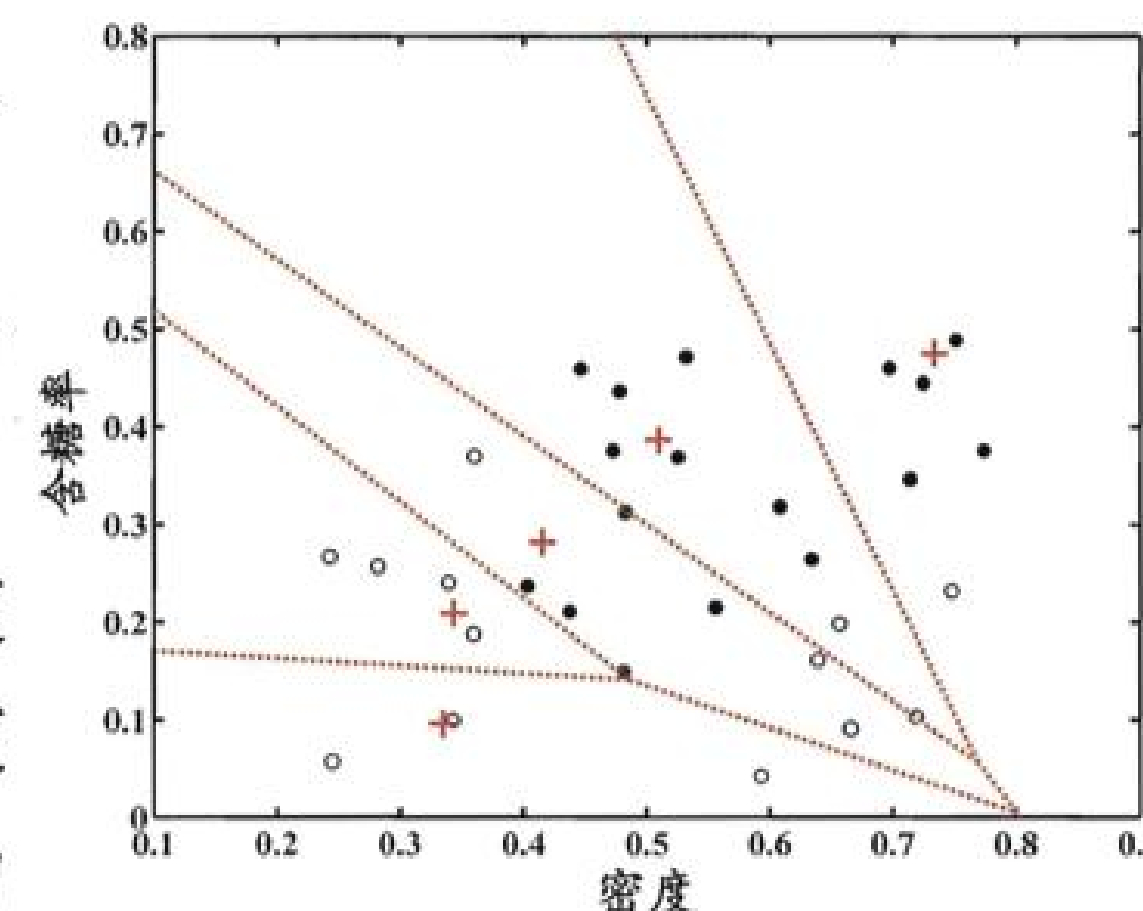
• LVQ举例

下面我们以后表 9.1 的西瓜数据集 4.0 为例来演示 LVQ 的学习过程. 令 9-21 号样本的类别标记为 c_2 , 其他样本的类别标记为 c_1 . 假定 $q = 5$, 即学习目标是找到 5 个原型向量 p_1, p_2, p_3, p_4, p_5 , 并假定其对应的类别标记分别为 c_1, c_2, c_2, c_1, c_1 .

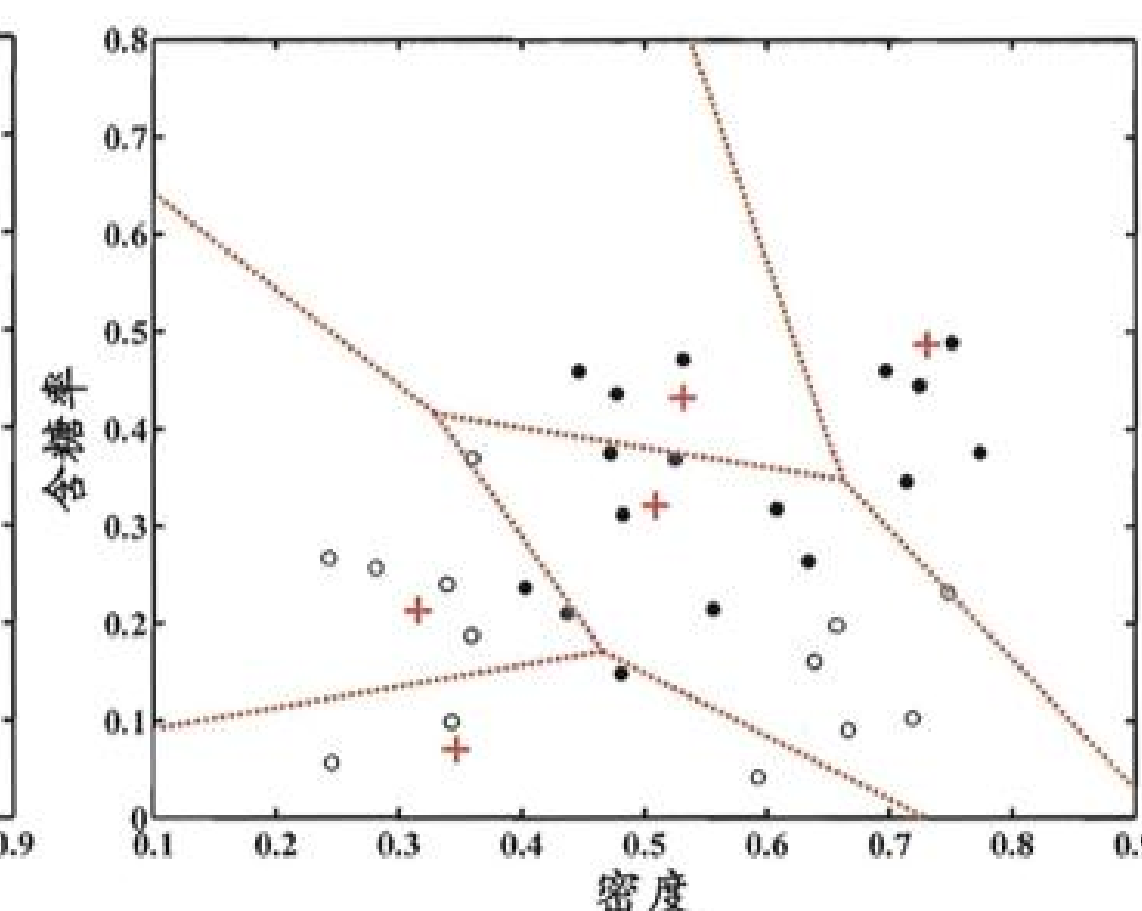
算法开始时, 根据样本的类别标记和簇的预设类别标记对原型向量进行随机初始化, 假定初始化为样本 $x_5, x_{12}, x_{18}, x_{23}, x_{29}$. 在第一轮迭代中, 假定随机选取的样本为 x_1 , 该样本与当前原型向量 p_1, p_2, p_3, p_4, p_5 的距离分别为 0.283, 0.506, 0.434, 0.260, 0.032. 由于 p_5 与 x_1 距离最近且两者具有相同的类别标记 c_2 , 假定学习率 $\eta = 0.1$, 则 LVQ 更新 p_5 得到新原型向量

$$\begin{aligned} p' &= p_5 + \eta \cdot (x_1 - p_5) \\ &= (0.725; 0.445) + 0.1 \cdot ((0.697; 0.460) - (0.725; 0.445)) \\ &= (0.722; 0.442) . \end{aligned}$$

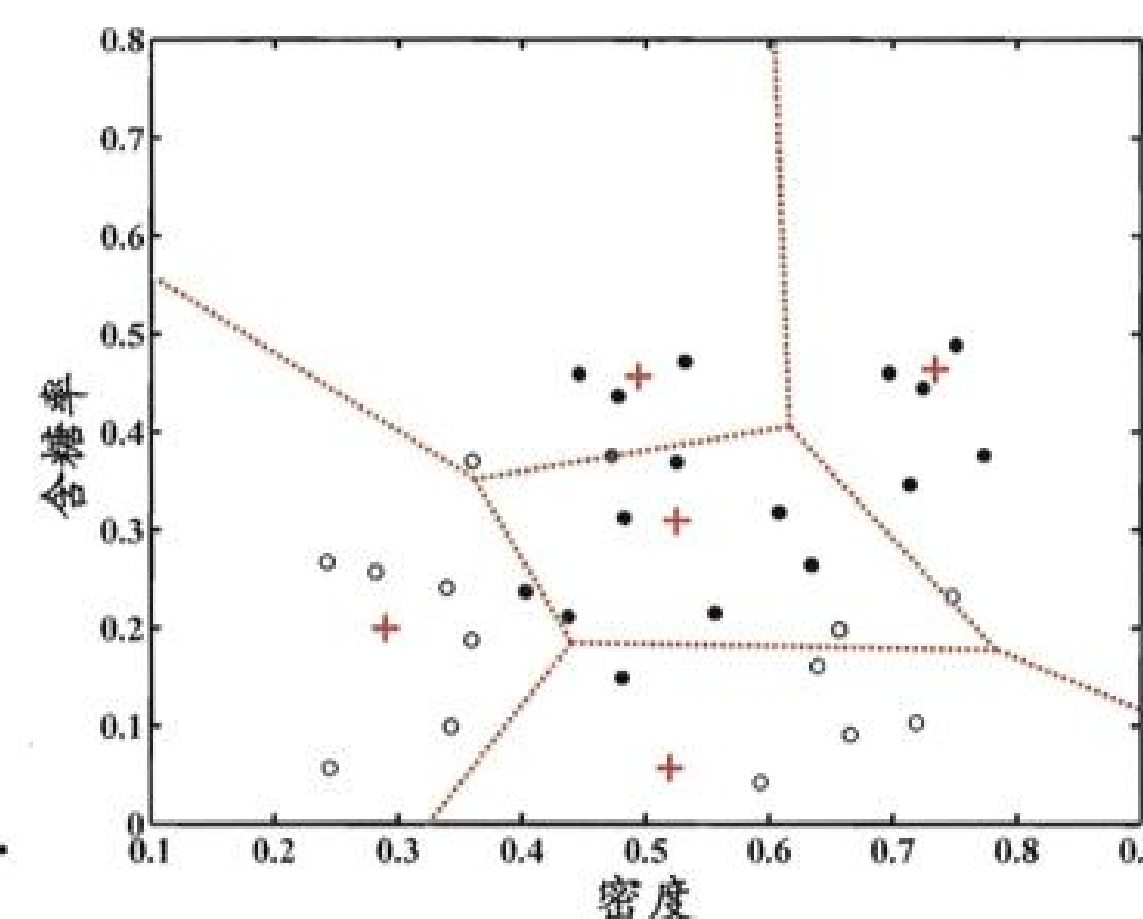
将 p_5 更新为 p' 后, 不断重复上述过程, 不同轮数之后的聚类结果如图 9.5 所示.



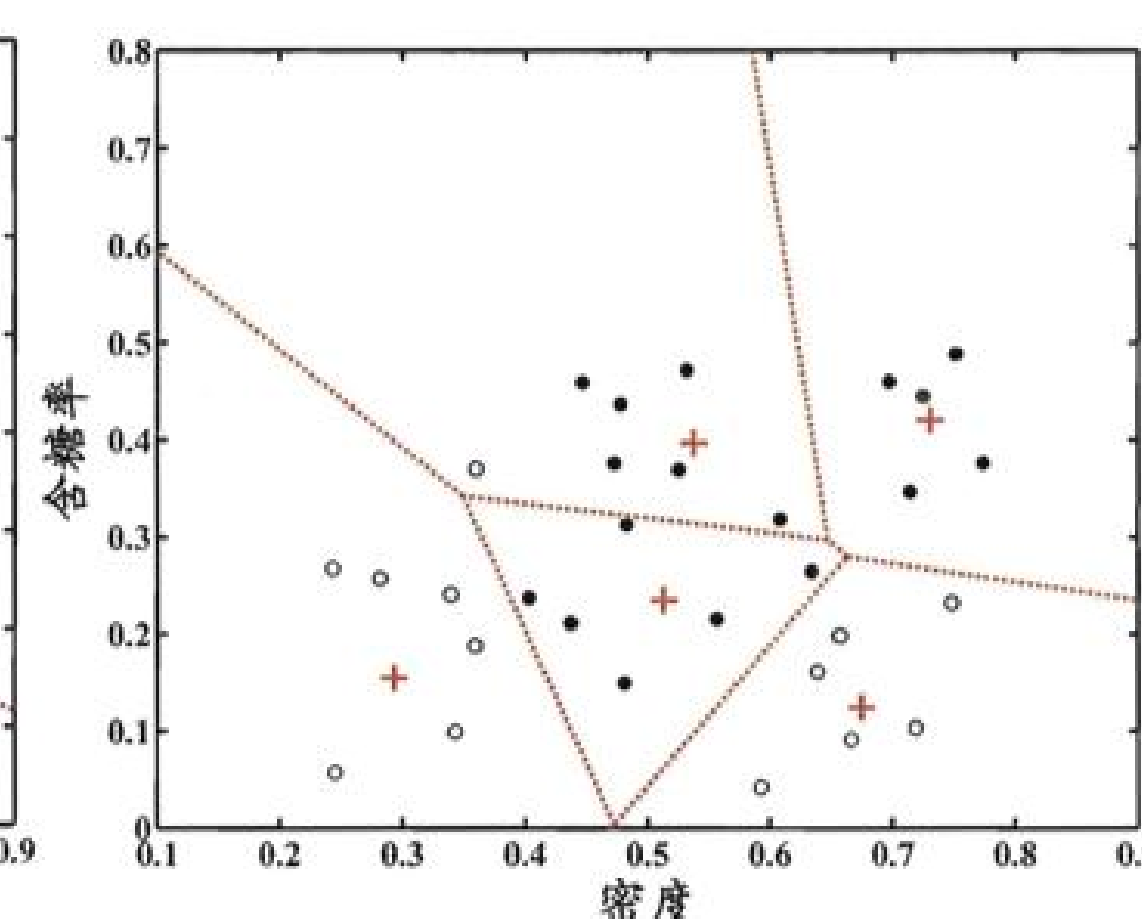
(a) 50 轮迭代后



(b) 100 轮迭代后



(c) 200 轮迭代后



(d) 400 轮迭代后

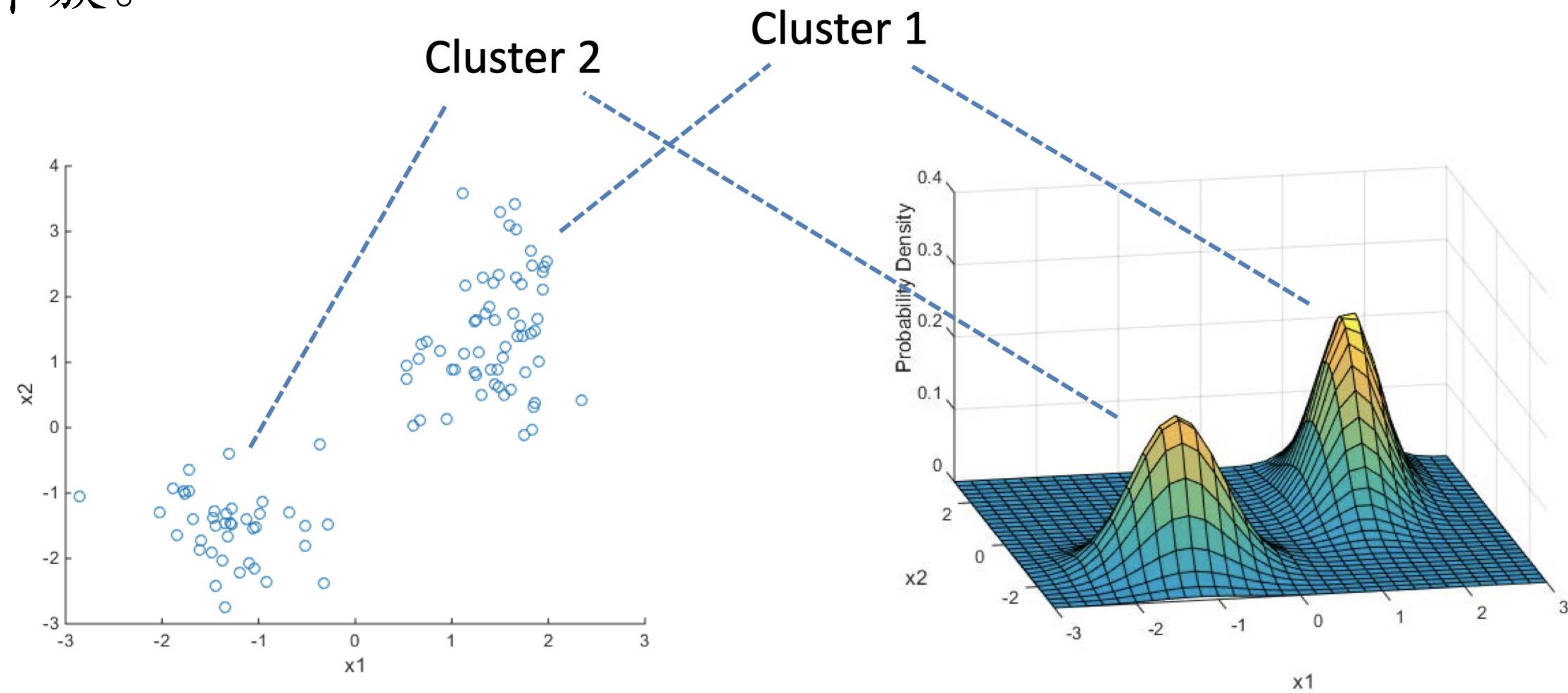
原型聚类- 高斯混合聚类

- 高斯混合聚类
 - 从概率的视角研究聚类算法，对数据聚类中不确定性建模
 - 采用概率模型来表达聚类原型，簇划分则由原型对应的后验概率确定
- 与K-Means区别与联系
 - K-Means 聚类将每个样本都精确地分配到某一个簇，类似于 k-means，高斯混合聚类模型同样需要预先指定聚簇的个数 K
 - 不同于 k-means，概率模型可表达每个样本来源的不确定性，即每个样本来自聚簇 c 的概率为 W_c ，其中 $c=1,\dots,k$ 。

原型聚类- 高斯混合聚类

- 概率模型

- 样本 x_i 是来自K个分布（成分）的混合独立同分布样本。混合中的每个成分就是一个簇。



将成分假设为任何分布，但是，正态分布是常见的建模选择 → 高斯混合模型

原型聚类- 高斯混合聚类

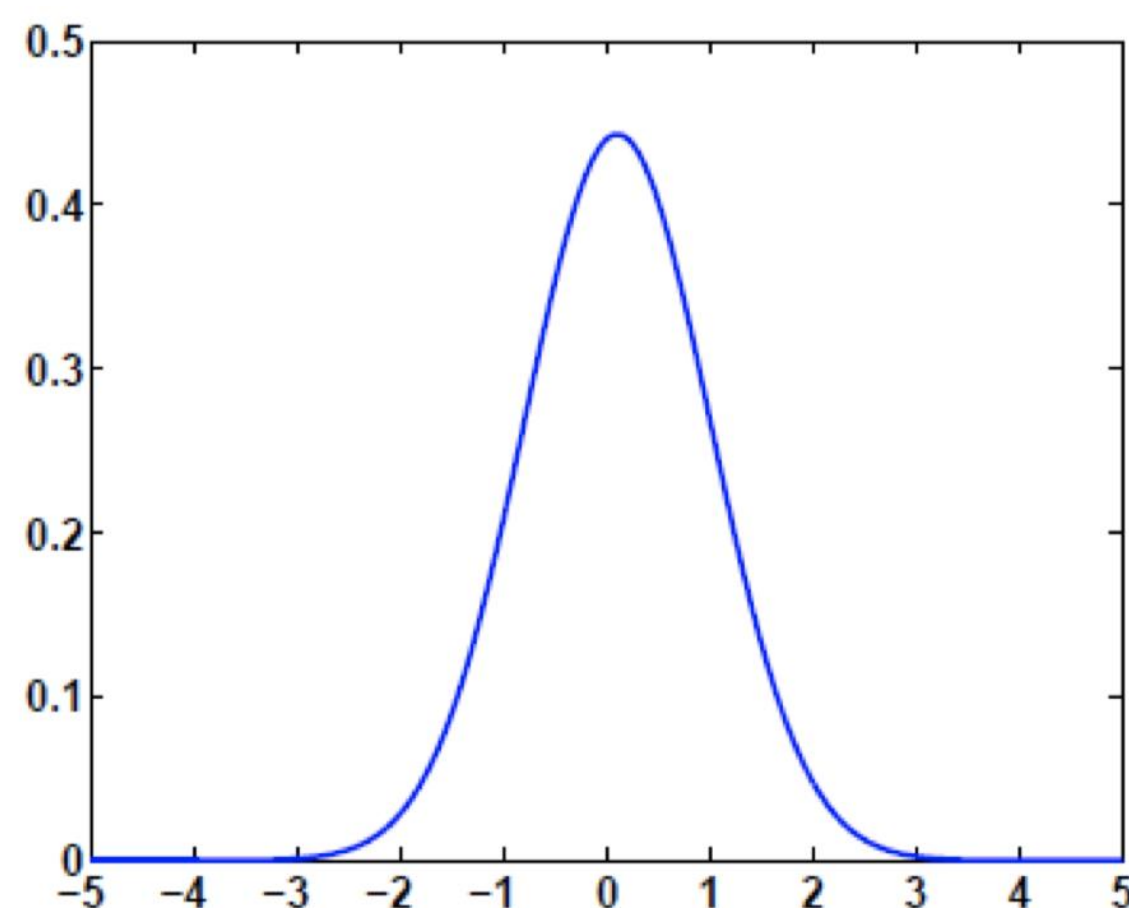
- 正态（高斯）分布

- 1-维高斯分布 $\mathcal{N}(x|\mu, \sigma) \equiv \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$

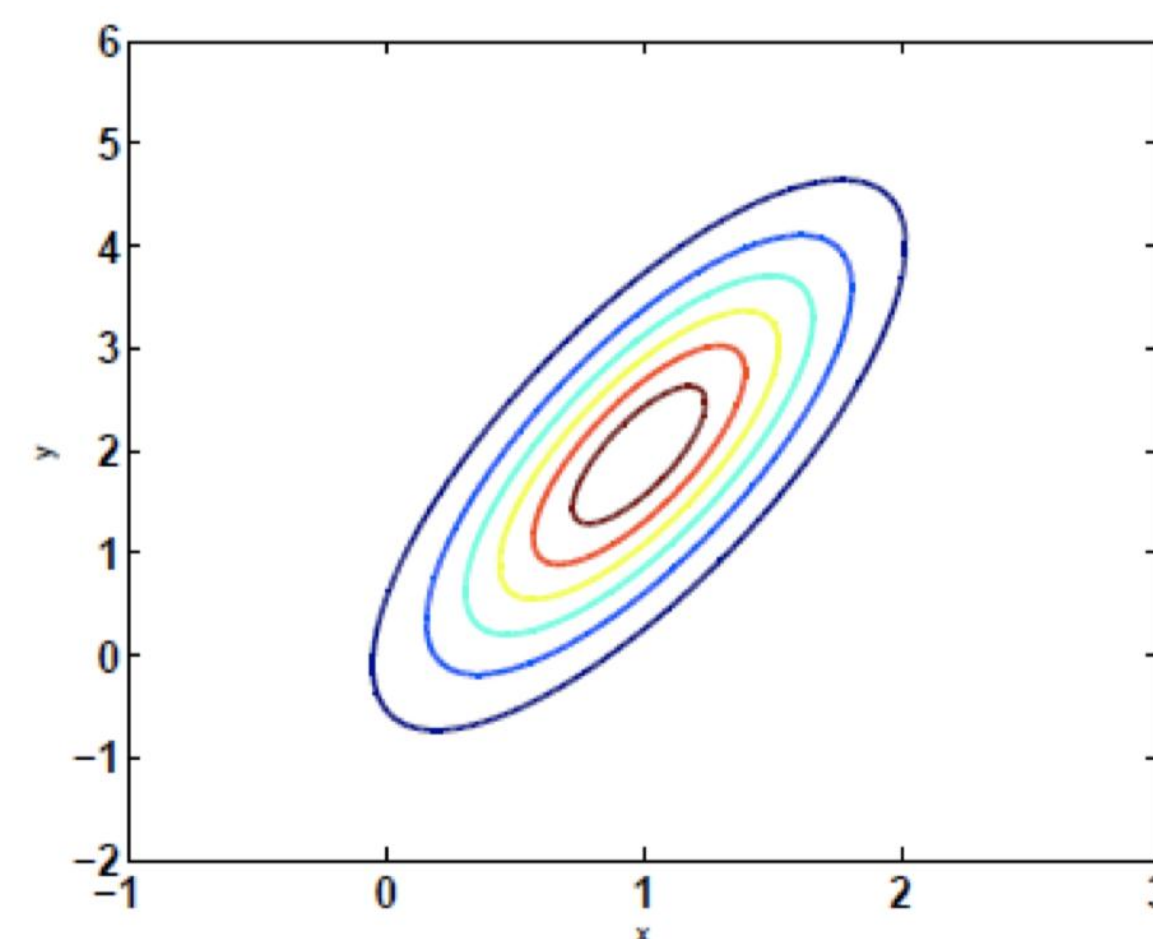
- d-维高斯分布 $\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) \equiv (2\pi)^{-\frac{d}{2}} |\boldsymbol{\Sigma}|^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$

- $\boldsymbol{\Sigma}$ 是协方差矩阵，是PSD（半正定）对称 $d \times d$ 矩阵

- $|\boldsymbol{\Sigma}|$ 表示行列式



(a) 1-Dim



(b) 2-Dim

原型聚类- 高斯混合聚类

- 多元高斯分布

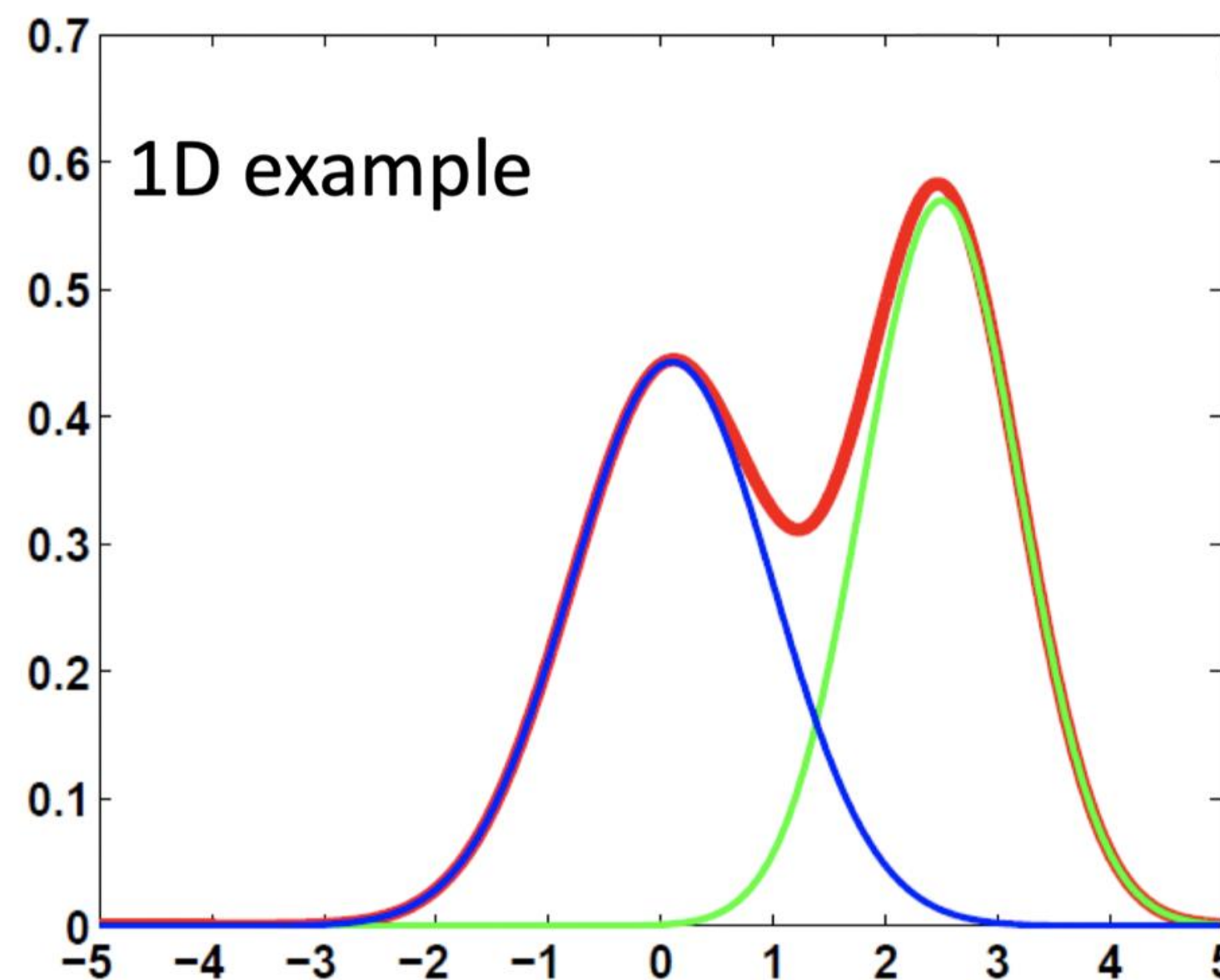
- 给定 n 维样本空间 X 中的随机向量 x ，若 x 服从高斯分布，其概率密度函数 $p(x|\mu, \Sigma)$ 为

- $$p(x) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

- 高斯混合分布：由 K 个混合成分组成，每个混合成分对应一个高斯分布

- $$p_{\mathcal{M}}(x) = \sum_{i=1}^k \alpha_i \cdot p(x | \mu_i, \Sigma_i)$$

- $\sum_{i=1}^k \alpha_i = 1$ ，混合系数； μ_i, Σ_i 是第 i 个高斯混合成分参数



原型聚类- 高斯混合聚类

- 聚类思想

- 假设样本数据服从混合高斯分布，根据样本数据集推出混合高斯分布的各个参数，以及各个样本最可能属于哪个高斯分布。GMM 的K个成分对应于聚类任务中的K个簇，每个样本划到最可能的高斯分布所对应的簇中。
- 假设K=2，则GMM的形式

$$p_{\mathcal{M}}(\mathbf{x}) = \alpha_1 \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) + \alpha_2 \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$$
$$\alpha_1 + \alpha_2 = 1$$

- 如何根据样本数据求出混合高斯分布的各个参数是关键所在？

原型聚类- 高斯混合聚类

- 样本的生成过程

- 由高斯混合分布给出：根据 $\alpha_1, \alpha_2, \dots, \alpha_k$ 定义的先验分布选择高斯混合成分，其中 α_i 为选择第 i 个混合成分的概率；根据被选择的混合成分的概率密度函数进行采样，从而生成相应的样本。

- 高斯混合聚类

- 若训练集 $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$ 根据上述过程生成，令随机变量 $z_j \in \{1, 2, \dots, k\}$ 表示生成样本 $\mathbf{x}_j$ 的高斯混合成分，其取值未知。显然， z_j 的先验概率 $P(z_j = i)$ 对应于 $\alpha_i (i = 1, 2, \dots, k)$ 。根据贝叶斯定理， z_j 后验分布：

$$p_{\mathcal{M}}(z_j = i | \mathbf{x}_j) = \frac{P(z_j = i) \cdot p_{\mathcal{M}}(\mathbf{x}_j | z_j = i)}{p_{\mathcal{M}}(\mathbf{x}_j)} = \frac{\alpha_i \cdot p(\mathbf{x}_j | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}{\sum_{l=1}^k \alpha_l \cdot p(\mathbf{x}_j | \boldsymbol{\mu}_l, \boldsymbol{\Sigma}_l)} \quad 9.30$$

后验分布相当于聚类模型

原型聚类- 高斯混合聚类

- 高斯混合聚类

- $p_M(z_j = i | x_j)$: γ_{ji} 表示样本 x_j 由第 i 个高斯混合成分生成的后验概率。

- 当高斯混合分布 (9.29) 已知时，高斯混合聚类将把样本集 D 划分为 k 个簇 $C = \{C_1, C_2, \dots, C_k\}$ ，每个样本 x_j 的簇标记 λ_j 如下确定：

- $$\lambda_j = \arg \max_{i \in \{1, 2, \dots, k\}} \gamma_{ji} \quad 9.31$$

- 从原型聚类角度来看，高斯混合聚类采用概率模型(高斯分布)对原型进行刻画，簇划分则由原型对应后验概率确定。

原型聚类- 高斯混合聚类

- 模型参数 $\{(\alpha_i, \mu_i, \Sigma) | 1 \leq i \leq k\}$ 如何求解?
- 给定样本集 D , 采用极大似然估计, 最大化对数似然:

$$LL(D) = \ln \left(\prod_{j=1}^m p_{\mathcal{M}}(\mathbf{x}_j) \right) \quad 9.32$$

- $$= \sum_{j=1}^m \ln \left(\sum_{i=1}^k \alpha_i \cdot p(\mathbf{x}_j | \mu_i, \Sigma_i) \right)$$

- 常采用 EM 算法进行迭代优化求解, 推导如下

如果知道每个样本是由哪个高斯混合成分生成, 则直接采用“极大似然估计”求解参数。

问题在于, 对于样本集 D , 我们并不知道每个样本点是由哪个高斯混合成分生成, 从而无法直接使用极大似然估计来求解参数。

原型聚类- 高斯混合聚类

• 模型参数如何求解?

若参数 $\{(\alpha_i, \mu_i, \Sigma_i) \mid 1 \leq i \leq k\}$ 能使式(9.32)最大化, 则由 $\frac{\partial LL(D)}{\partial \mu_i} = 0$ 有

$$\sum_{j=1}^m \frac{\alpha_i \cdot p(\mathbf{x}_j \mid \mu_i, \Sigma_i)}{\sum_{l=1}^k \alpha_l \cdot p(\mathbf{x}_j \mid \mu_l, \Sigma_l)} (\mathbf{x}_j - \mu_i) = 0, \quad (9.33)$$

由式(9.30)以及 $\gamma_{ji} = p_{\mathcal{M}}(z_j = i \mid \mathbf{x}_j)$, 有

$$\mu_i = \frac{\sum_{j=1}^m \gamma_{ji} \mathbf{x}_j}{\sum_{j=1}^m \gamma_{ji}}, \quad (9.34)$$

即各混合成分的均值可通过样本加权平均来估计, 样本权重是每个样本属于该成分的后验概率. 类似的, 由 $\frac{\partial LL(D)}{\partial \Sigma_i} = 0$ 可得

$$\Sigma_i = \frac{\sum_{j=1}^m \gamma_{ji} (\mathbf{x}_j - \mu_i)(\mathbf{x}_j - \mu_i)^T}{\sum_{j=1}^m \gamma_{ji}}. \quad (9.35)$$

对于混合系数 α_i , 除了要最大化 $LL(D)$, 还需满足 $\alpha_i \geq 0, \sum_{i=1}^k \alpha_i = 1$. 考虑 $LL(D)$ 的拉格朗日形式

$$LL(D) + \lambda \left(\sum_{i=1}^k \alpha_i - 1 \right), \quad (9.36)$$

其中 λ 为拉格朗日乘子. 由式(9.36)对 α_i 的导数为 0, 有

$$\sum_{j=1}^m \frac{p(\mathbf{x}_j \mid \mu_i, \Sigma_i)}{\sum_{l=1}^k \alpha_l \cdot p(\mathbf{x}_j \mid \mu_l, \Sigma_l)} + \lambda = 0, \quad (9.37)$$

两边同乘以 α_i , 对所有样本求和可知 $\lambda = -m$, 有

$$\alpha_i = \frac{1}{m} \sum_{j=1}^m \gamma_{ji}, \quad (9.38)$$

即每个高斯成分的混合系数由样本属于该成分的平均后验概率确定.

由上述推导即可获得高斯混合模型的 EM 算法: 在每步迭代中, 先根据当前参数来计算每个样本属于每个高斯成分的后验概率 γ_{ji} (E步), 再根据式(9.34)、(9.35)和(9.38)更新模型参数 $\{(\alpha_i, \mu_i, \Sigma_i) \mid 1 \leq i \leq k\}$ (M步).

原型聚类- 高斯混合聚类

输入: 样本集 $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$;
高斯混合成分个数 k .

过程:

```
1: 初始化高斯混合分布的模型参数  $\{(\alpha_i, \mu_i, \Sigma_i) \mid 1 \leq i \leq k\}$ 
2: repeat
3:   for  $j = 1, 2, \dots, m$  do
4:     根据式(9.30)计算  $\mathbf{x}_j$  由各混合成分生成的后验概率, 即
        $\gamma_{ji} = p_{\mathcal{M}}(z_j = i \mid \mathbf{x}_j) \ (1 \leq i \leq k)$ 
5:   end for
6:   for  $i = 1, 2, \dots, k$  do
7:     计算新均值向量:  $\mu'_i = \frac{\sum_{j=1}^m \gamma_{ji} \mathbf{x}_j}{\sum_{j=1}^m \gamma_{ji}}$ ;
8:     计算新协方差矩阵:  $\Sigma'_i = \frac{\sum_{j=1}^m \gamma_{ji} (\mathbf{x}_j - \mu'_i)(\mathbf{x}_j - \mu'_i)^T}{\sum_{j=1}^m \gamma_{ji}}$ ;
9:     计算新混合系数:  $\alpha'_i = \frac{\sum_{j=1}^m \gamma_{ji}}{m}$ ;
10:  end for
11: 将模型参数  $\{(\alpha_i, \mu_i, \Sigma_i) \mid 1 \leq i \leq k\}$  更新为  $\{(\alpha'_i, \mu'_i, \Sigma'_i) \mid 1 \leq i \leq k\}$ 
12: until 满足停止条件
13:  $C_i = \emptyset \ (1 \leq i \leq k)$ 
14: for  $j = 1, 2, \dots, m$  do
15:   根据式(9.31)确定  $\mathbf{x}_j$  的簇标记  $\lambda_j$ ;
16:   将  $\mathbf{x}_j$  划入相应的簇:  $C_{\lambda_j} = C_{\lambda_j} \cup \{\mathbf{x}_j\}$ 
17: end for
```

输出: 簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

1. E-step: 求期望 $E(\gamma_{jk} \mid X, \theta)$ for all $j = 1, 2, \dots, N$

2. M-step: 求极大, 计算新一轮迭代的模型参数

- 首先初始化参数

- E-step: 依据当前参数, 计算每个数据 j 来自子模型 k 的可能性

$$\gamma_{jk} = \frac{\alpha_k \phi(\mathbf{x}_j \mid \theta_k)}{\sum_{k=1}^K \alpha_k \phi(\mathbf{x}_j \mid \theta_k)}, j = 1, 2, \dots, N; k = 1, 2, \dots, K$$

- M-step: 计算新一轮迭代的模型参数

$$\mu_k = \frac{\sum_j^N (\gamma_{jk} \mathbf{x}_j)}{\sum_j^N \gamma_{jk}}, k = 1, 2, \dots, K$$

$$\Sigma_k = \frac{\sum_j^N \gamma_{jk} (\mathbf{x}_j - \mu_k)(\mathbf{x}_j - \mu_k)^T}{\sum_j^N \gamma_{jk}}, k = 1, 2, \dots, K \quad (\text{用这一轮更新后的 } \mu_k)$$

$$\alpha_k = \frac{\sum_{j=1}^N \gamma_{jk}}{N}, k = 1, 2, \dots, K$$

原型聚类- 高斯混合聚类

• 举例

以表 9.1 的西瓜数据集 4.0 为例, 令高斯混合成分的个数 $k = 3$. 算法开始时, 假定将高斯混合分布的模型参数初始化为: $\alpha_1 = \alpha_2 = \alpha_3 = \frac{1}{3}$; $\mu_1 = x_6$, $\mu_2 = x_{22}$, $\mu_3 = x_{27}$; $\Sigma_1 = \Sigma_2 = \Sigma_3 = \begin{pmatrix} 0.1 & 0.0 \\ 0.0 & 0.1 \end{pmatrix}$.

在第一轮迭代中, 先计算样本由各混合成分生成的后验概率. 以 x_1 为例, 由式(9.30)算出后验概率 $\gamma_{11} = 0.219$, $\gamma_{12} = 0.404$, $\gamma_{13} = 0.377$. 所有样本的后验概率算完后, 得到如下新的模型参数:

$$\alpha'_1 = 0.361, \alpha'_2 = 0.323, \alpha'_3 = 0.316$$

$$\mu'_1 = (0.491; 0.251), \mu'_2 = (0.571; 0.281), \mu'_3 = (0.534; 0.295)$$

$$\Sigma'_1 = \begin{pmatrix} 0.025 & 0.004 \\ 0.004 & 0.016 \end{pmatrix}, \Sigma'_2 = \begin{pmatrix} 0.023 & 0.004 \\ 0.004 & 0.017 \end{pmatrix}, \Sigma'_3 = \begin{pmatrix} 0.024 & 0.005 \\ 0.005 & 0.016 \end{pmatrix}$$

模型参数更新后, 不断重复上述过程, 不同轮数之后的聚类结果如图 9.7 所示.

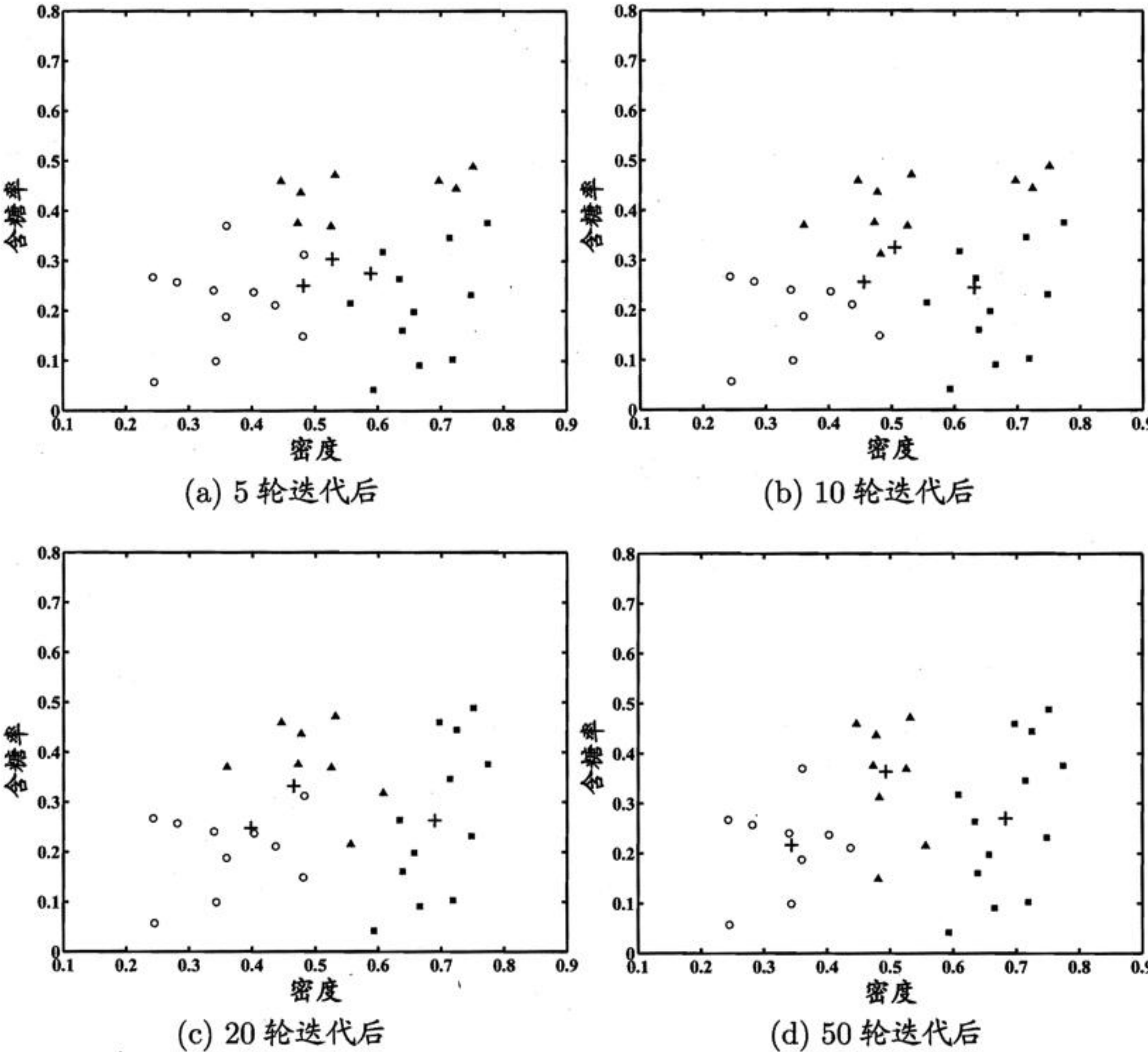


图 9.7 高斯混合聚类($k = 3$)在不同轮数迭代后的聚类结果. 其中样本簇 C_1 , C_2 与 C_3 中的样本点分别用“o”, “■”与“▲”表示, 各高斯混合成分的均值向量用“+”表示.

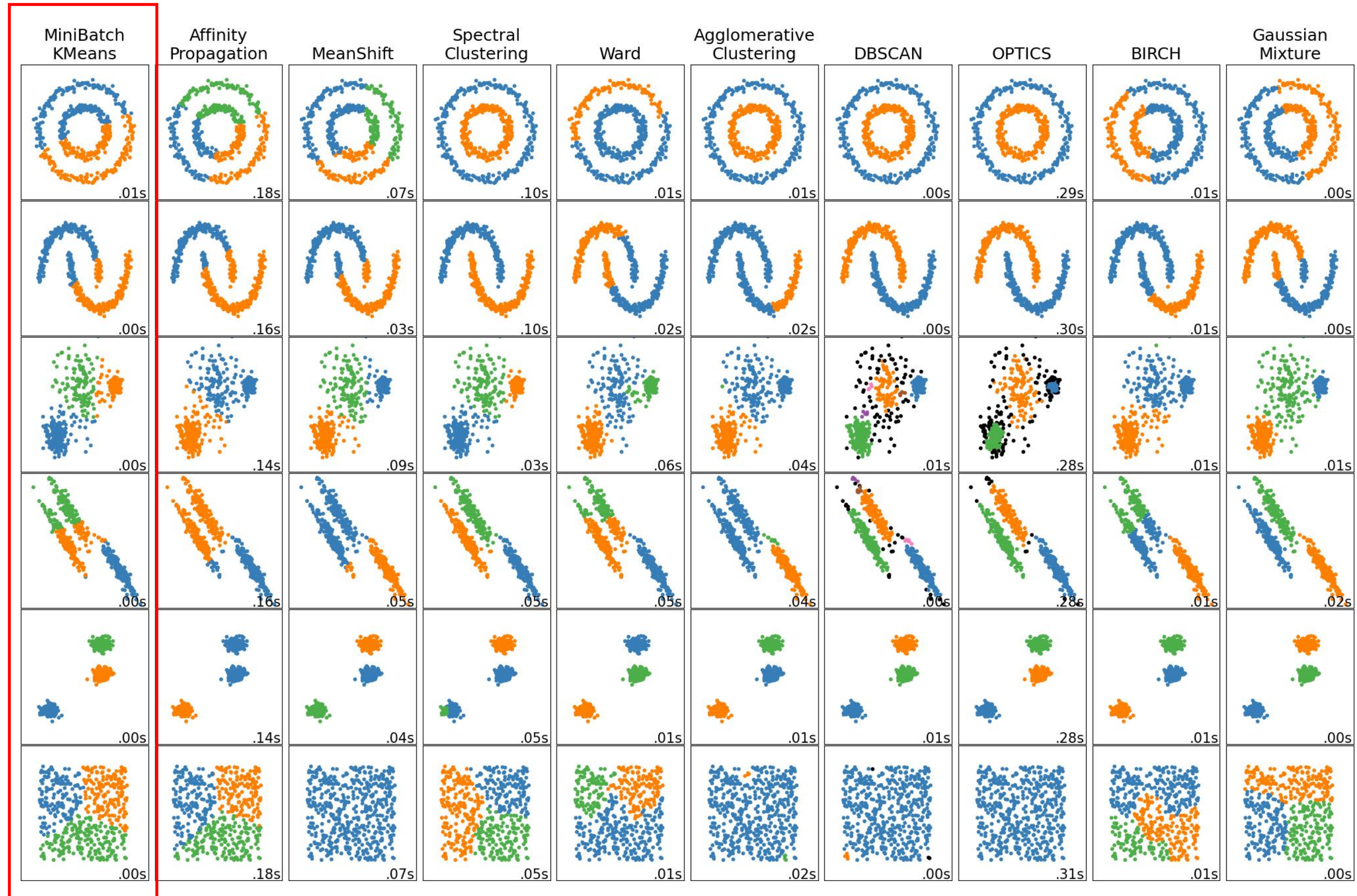
原型聚类- 高斯混合聚类

- 高斯混合聚类与K-Mean区别和联系
 - 联系：GMM和K-means算法都是迭代算法，两者先对参数赋初值，然后交替执行两个步骤，一个步骤是对数据的估计（K-means是每个点所属簇，GMM是隐含变量的期望）；另一个步骤是用上一步算出的估计值重新计算参数值，更新待估计参数（K-means是簇心位置，GMM是混合系数和各个高斯分布的中心位置和协方差矩阵）。
 - 区别：和K-means聚类算法不同（它直接给出样本点的类别），GMM还可以给出某个样本点属于某个类别的概率，这种方式提供的信息量显然更多。

内 容 大 纲

- 无监督学习
- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

K-Means 聚类存在问题



密度聚类 (Density-based Clustering)

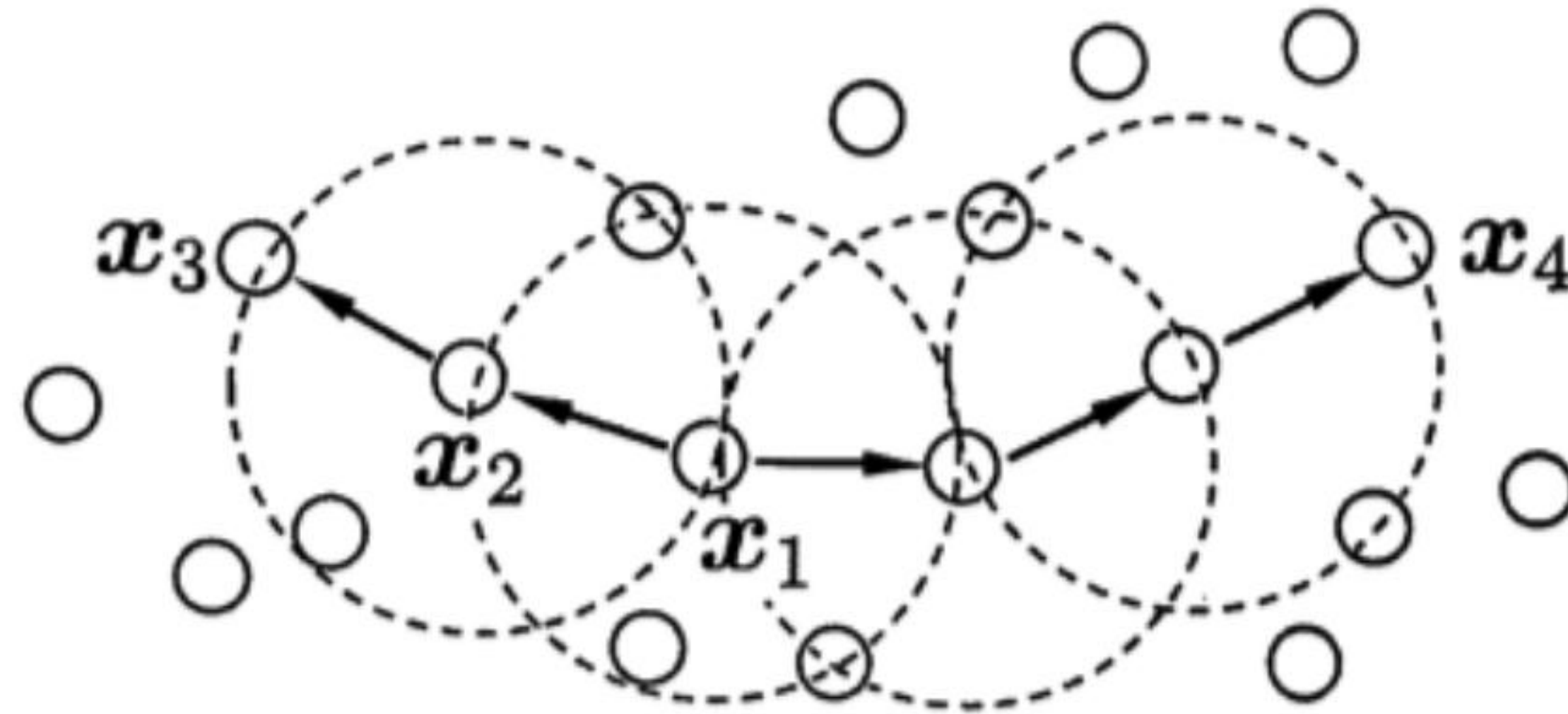
- 算法假设：聚类结构通过样本分布的紧密程度刻画
- 从样本密度的角度来考察样本之间的可连接性，依据可连接样本不断扩展聚类簇以获得最终的聚类结果
- DBSCAN
 - Density-based Spatial Clustering of Applications with Noise
 - 著名密度聚类算法
 - “邻域” (neighborhood) 参数 (ϵ , MinPts) 刻画样本分布的紧密程度

密度聚类

- 给定数据集 $D = \{x_1, x_2, \dots, x_m\}$, 概念定义如下:
 - ϵ -邻域: 给定任意 $x_j \in D$, $N_\epsilon(x_j) = \{x_i \in D \mid \text{dist}(x_i, x_j) \leq \epsilon\}$
 - 核心对象: 若 $|N_\epsilon(x_j)| \geq \text{MinPts}$, 则 x_j 是一个核心对象
 - 密度直达: 若 x_j 位于 x_i 的 ϵ 领域中, 且 x_i 是核心对象, 则 x_j 由 x_i 密度直达
 - 密度可达: 对 x_i 与 x_j , 若存在样本序列 $p_1, p_2, \dots, p_n$, 其中 $p_1 = x_i$, $p_n = x_j$ 且 p_{i+1} 由 p_i 密度直达, 则称 x_j 由 x_i 密度可达;
 - 密度相连: 对 x_i 与 x_j , 若存在 x_k 使得 x_i 与 x_j 均由 x_k 密度可达, 则称 x_i 与 x_j 密度相连

密度聚类

- DBSCAN直观例子($\text{MinPts}=3$)

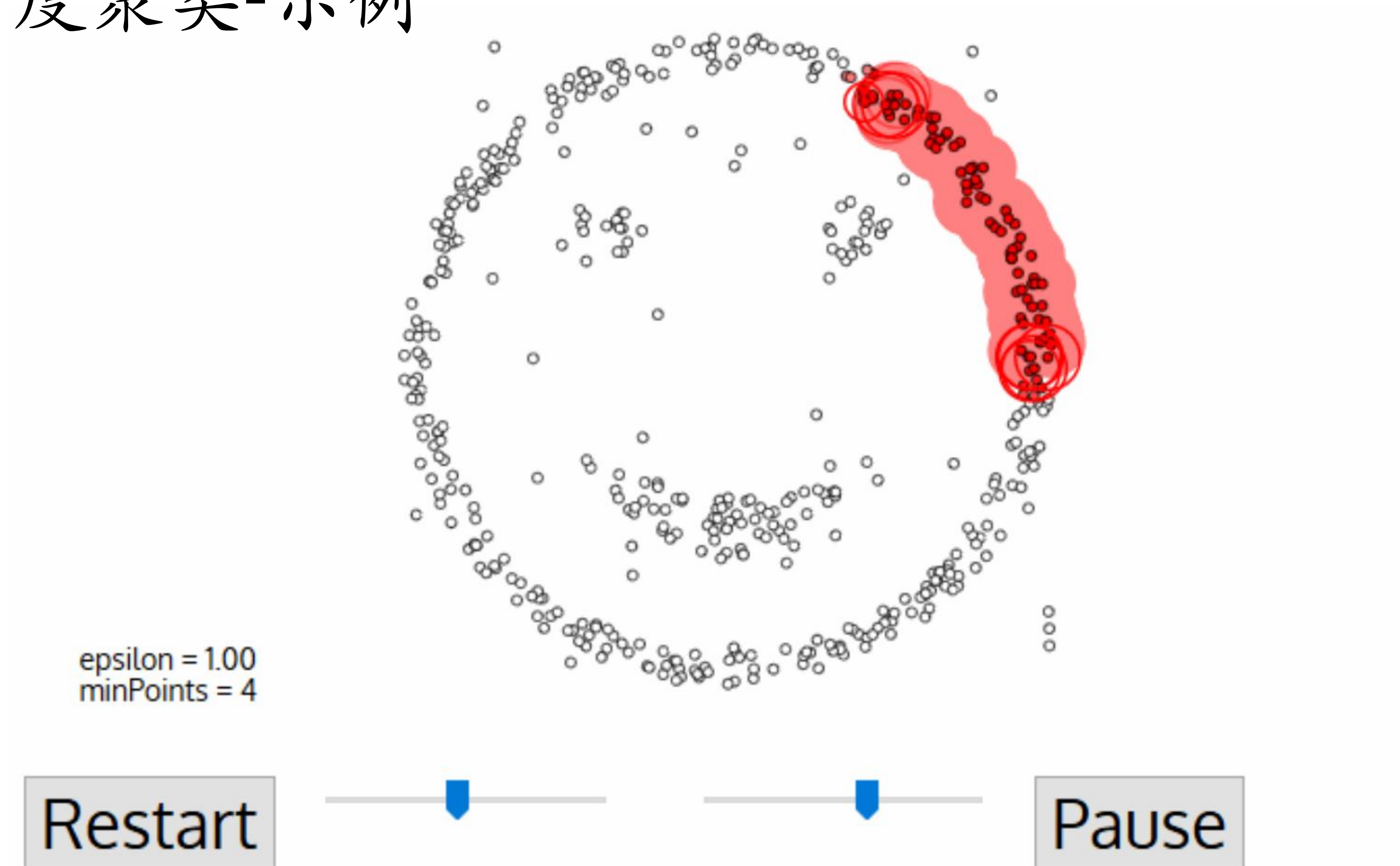


- 虚线显示出 ϵ -邻域, x_1 是核心对象, x_2 由 x_1 密度直达, x_3 由 x_1 密度可达, x_3 与 x_4 密度相连
- DBSCAN的“簇”定义
 - 由密度可达关系导出的最大的密度相连样本集合

密度聚类

- 给定邻域参数，簇 $C \subseteq D$ 是满足以下性质的非空样本子集
 - 连接性： $x_i \in C, x_j \in C \Rightarrow x_i$ 与 x_j 密度相连
 - 最大性： $x_i \in C, x_j$ 由 x_i 密度可达 $\Rightarrow x_j \in C$
- 如何从数据集 D 中找出满足以上性质的 **聚类簇** 呢?
 - 若 x 为核心对象，由 x 密度可达的所有样本组成的集合，记为 $X = \{x' \in D | x' \text{ 由 } x \text{ 密度可达}\}$ ，则 X 为满足连接性与最大性的簇。

密度聚类-示例



密度聚类

• DBSCAN算法

大体思想：1) 根据邻域参数确定核心对象；2) 任选数据集中一个核心对象作为种子，再由核心对象出发确定相应的聚类簇

输入：样本集 $D = \{x_1, x_2, \dots, x_m\}$;
邻域参数 $(\epsilon, MinPts)$.

过程:

```
1: 初始化核心对象集合:  $\Omega = \emptyset$ 
2: for  $j = 1, 2, \dots, m$  do
3:   确定样本  $x_j$  的  $\epsilon$ -邻域  $N_\epsilon(x_j)$ ;
4:   if  $|N_\epsilon(x_j)| \geq MinPts$  then
5:     将样本  $x_j$  加入核心对象集合:  $\Omega = \Omega \cup \{x_j\}$ 
6:   end if
7: end for
8: 初始化聚类簇数:  $k = 0$ 
9: 初始化未访问样本集合:  $\Gamma = D$ 
10: while  $\Omega \neq \emptyset$  do 每次生成一个簇
11:   记录当前未访问样本集合:  $\Gamma_{old} = \Gamma$ ;
12:   随机选取一个核心对象  $o \in \Omega$ , 初始化队列  $Q = \langle o \rangle$ ;
13:    $\Gamma = \Gamma \setminus \{o\}$ ;
14:   while  $Q \neq \emptyset$  do
15:     取出队列  $Q$  中的首个样本  $q$ ;
16:     if  $|N_\epsilon(q)| \geq MinPts$  then
17:       令  $\Delta = N_\epsilon(q) \cap \Gamma$ ;
18:       将  $\Delta$  中的样本加入队列  $Q$ ;
19:        $\Gamma = \Gamma \setminus \Delta$ ;
20:     end if
21:   end while
22:    $k = k + 1$ , 生成聚类簇  $C_k = \Gamma_{old} \setminus \Gamma$ ;
23:    $\Omega = \Omega \setminus C_k$ 
24: end while
```

输出：簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

根据给定的邻域参数确定
所有的核心对象

以任一核心对象为出发，找出
由其密度可达的所有样本生成
聚类簇；
利用队列数据结构，访问满足
密度可达所有的样本。

密度聚类

• DBSCAN 举例

以表 9.1 的西瓜数据集 4.0 为例, 假定邻域参数 $(\epsilon, MinPts)$ 设置为 $\epsilon = 0.11$, $MinPts = 5$. DBSCAN 算法先找出各样本的 ϵ -邻域并确定核心对象集合: $\Omega = \{x_3, x_5, x_6, x_8, x_9, x_{13}, x_{14}, x_{18}, x_{19}, x_{24}, x_{25}, x_{28}, x_{29}\}$. 然后, 从 Ω 中随机选取一个核心对象作为种子, 找出由它密度可达的所有样本, 这就构成了第一个聚类簇. 不失一般性, 假定核心对象 x_8 被选中作为种子, 则 DBSCAN 生成的第一个聚类簇为

$$C_1 = \{x_6, x_7, x_8, x_{10}, x_{12}, x_{18}, x_{19}, x_{20}, x_{23}\}.$$

然后, DBSCAN 将 C_1 中包含的核心对象从 Ω 中去除: $\Omega = \Omega \setminus C_1 = \{x_3, x_5, x_9, x_{13}, x_{14}, x_{24}, x_{25}, x_{28}, x_{29}\}$. 再从更新后的集合 Ω 中随机选取一个核心对象作为种子来生成下一个聚类簇. 上述过程不断重复, 直至 Ω 为空. 图 9.10 显示出 DBSCAN 先后生成聚类簇的情况. C_1 之后生成的聚类簇为

$$C_2 = \{x_3, x_4, x_5, x_9, x_{13}, x_{14}, x_{16}, x_{17}, x_{21}\};$$

$$C_3 = \{x_1, x_2, x_{22}, x_{26}, x_{29}\};$$

$$C_4 = \{x_{24}, x_{25}, x_{27}, x_{28}, x_{30}\}.$$

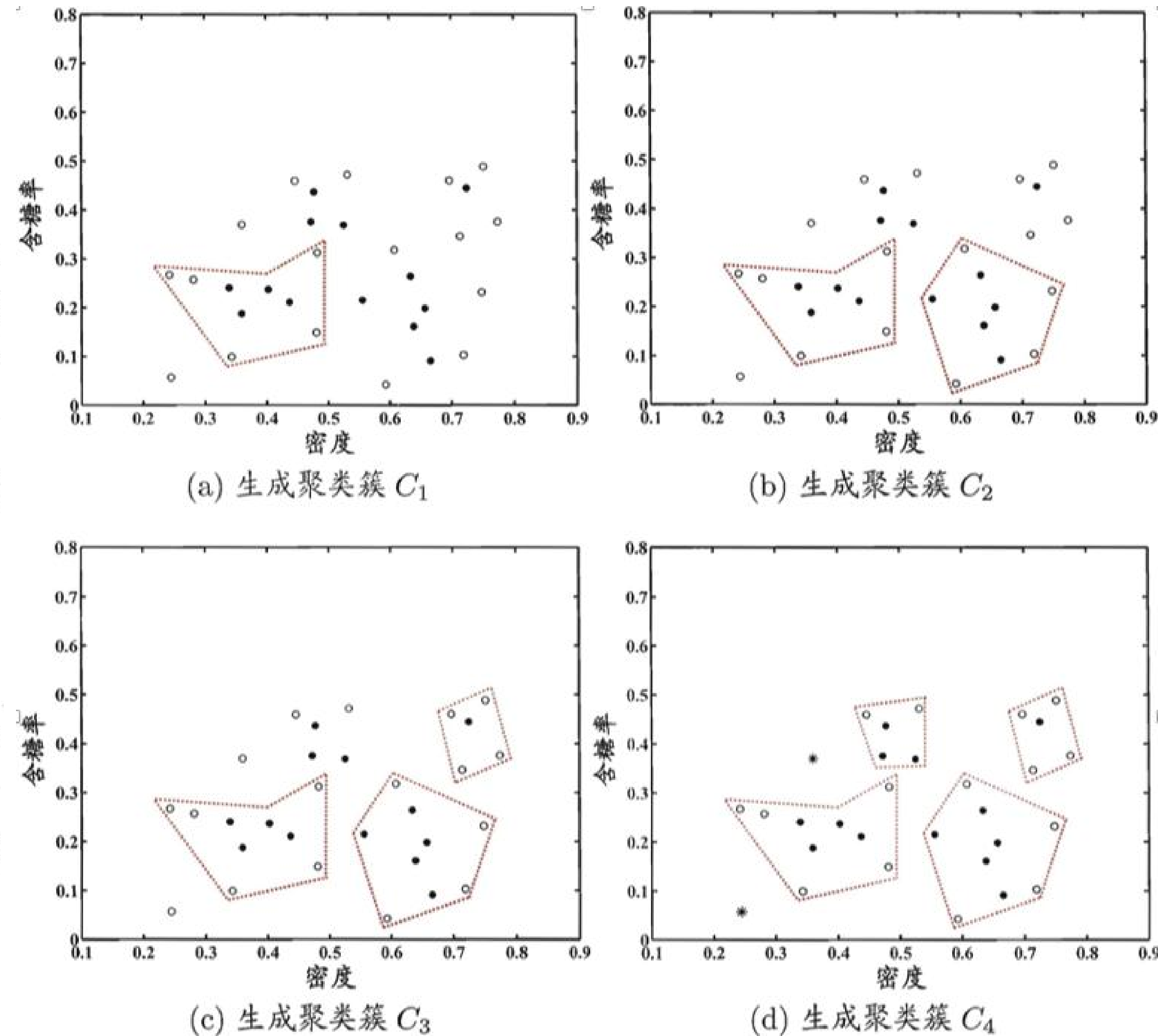


图 9.10 DBSCAN 算法($\epsilon = 0.11$, $MinPts = 5$)生成聚类簇的先后情况. 核心对象、非核心对象、噪声样本分别用“●”“○”“*”表示, 红色虚线显示出簇划分.

内 容 大 纲

- 无监督学习
- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

层次聚类

- 层次聚类(hierarchical clustering)
 - 在不同层次对数据集进行划分，从而形成树形的聚类结构
 - 数据集的划分可采用"自底向上"的聚合策略，也可采用"自顶向下"的分拆策略.
- AGNES (AGglomerative NESting)
 - 一种采用自底向上聚合策略的层次聚类算法
 - 先将数据集中的每个样本看作一个初始聚类簇，然后在算法运行的每一步中将距离最近的两个聚类簇进行合并，该过程不断重复，直至达到预设的聚类簇个数

层次聚类

- 如何计算聚类簇之间的距离
 - 给定聚类簇 C_i 与 C_j 可通过下面的式子来计算距离:

- 最小距离: 由两个簇的最近样本决定

$$d_{\min}(C_i, C_j) = \min_{x \in C_i, z \in C_j} \text{dist}(x, z)$$

- 最大距离: 由两个簇的最远样本决定

$$d_{\max}(C_i, C_j) = \max_{x \in C_i, z \in C_j} \text{dist}(x, z)$$

- 平均距离: 由两个簇的所有样本共同决定

$$d_{\text{avg}}(C_i, C_j) = \frac{1}{|C_i| |C_j|} \sum_{x \in C_i} \sum_{z \in C_j} \text{dist}(x, z)$$

层次聚类

• 层次聚类算法

大体思想：将距离最近的两个聚类簇进行合并

输入：样本集 $D = \{x_1, x_2, \dots, x_m\}$;
聚类簇距离度量函数 d ;
聚类簇数 k .

过程：

```
1: for  $j = 1, 2, \dots, m$  do
2:    $C_j = \{x_j\}$ 
3: end for
4: for  $i = 1, 2, \dots, m$  do
5:   for  $j = 1, 2, \dots, m$  do
6:      $M(i, j) = d(C_i, C_j)$ ;
7:      $M(j, i) = M(i, j)$ 
8:   end for
9: end for
10: 设置当前聚类簇个数:  $q = m$ 
11: while  $q > k$  do
12:   找出距离最近的两个聚类簇  $C_{i^*}$  和  $C_{j^*}$ ;
13:   合并  $C_{i^*}$  和  $C_{j^*}$ :  $C_{i^*} = C_{i^*} \cup C_{j^*}$ ;
14:   for  $j = j^* + 1, j^* + 2, \dots, q$  do
15:     将聚类簇  $C_j$  重编号为  $C_{j-1}$ 
16:   end for
17:   删除距离矩阵  $M$  的第  $j^*$  行与第  $j^*$  列;
18:   for  $j = 1, 2, \dots, q - 1$  do
19:      $M(i^*, j) = d(C_{i^*}, C_j)$ ;
20:      $M(j, i^*) = M(i^*, j)$ 
21:   end for
22:    $q = q - 1$ 
23: end while
输出：簇划分  $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$ 
```

先将每个样本当作初始化簇

矩阵M表示聚类簇距离矩阵

AGNES 不断合并距离最近的聚类簇，并对合并得到的聚类簇的距离矩阵进行更新;

更新矩阵M

层次聚类

• 层次聚类举例

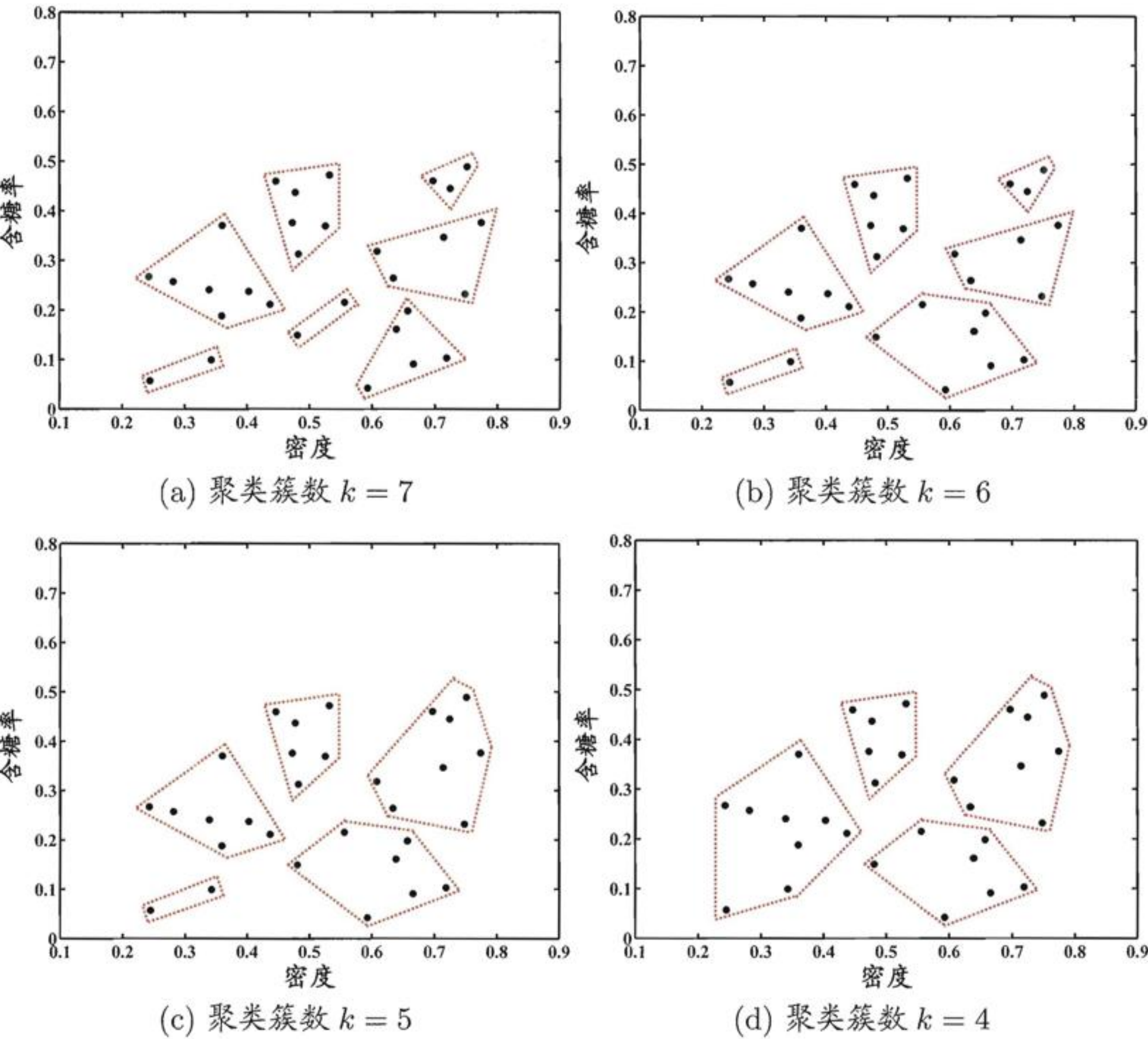


图 9.13 西瓜数据集 4.0 上 AGNES 算法(采用 $d_{\max}$)在不同聚类簇数($k = 7, 6, 5, 4$)时的簇划分结果. 样本点用“•”表示, 红色虚线显示出簇划分.

以西瓜数据集 4.0 为例, 令 AGNES 算法一直执行到所有样本出现在同一个簇中, 即 $k = 1$, 则可得到图 9.12 所示的“树状图” (dendrogram), 其中每层链接一组聚类簇.

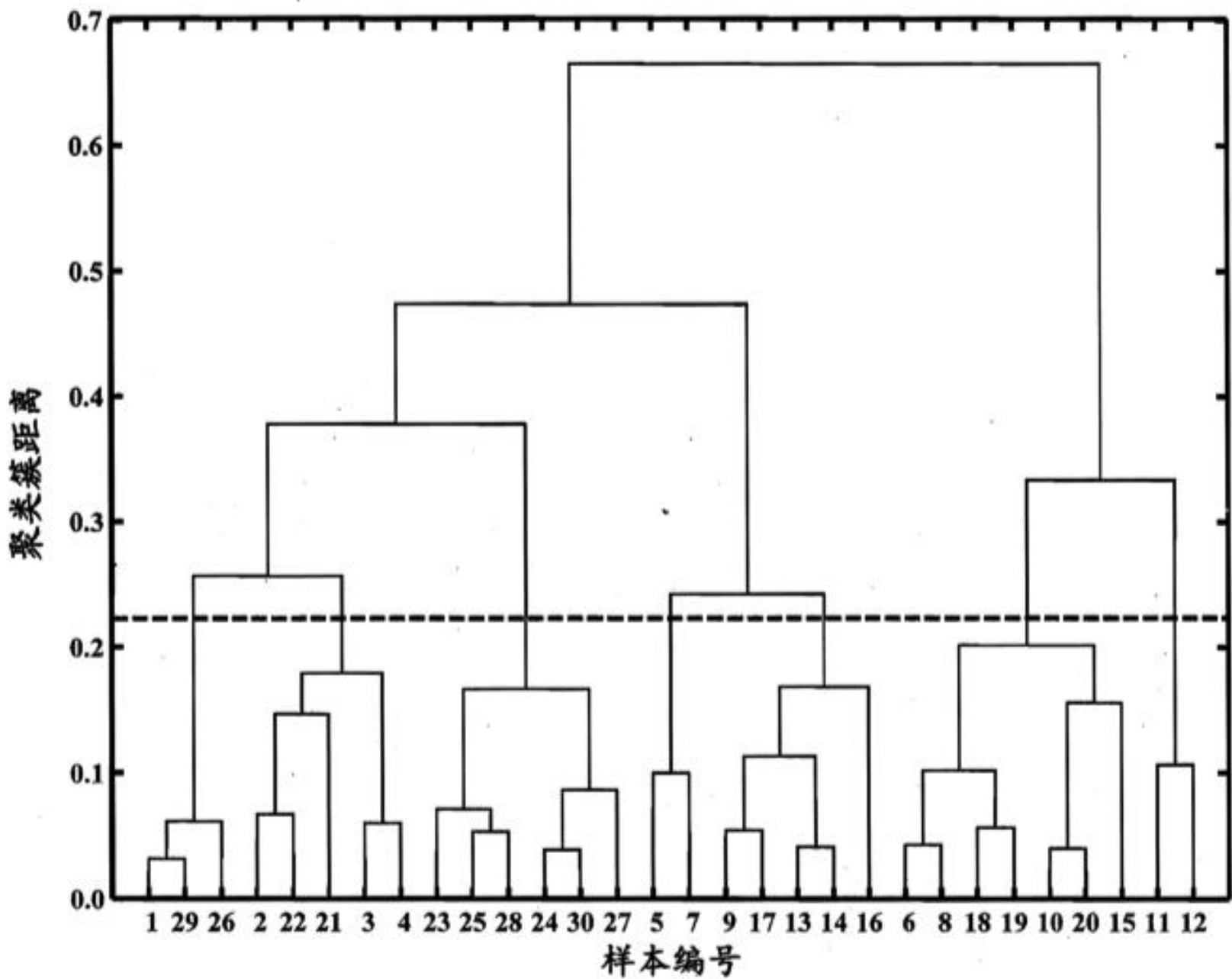


图 9.12 西瓜数据集 4.0 上 AGNES 算法生成的树状图(采用 $d_{\max}$). 横轴对应于样本编号, 纵轴对应于聚类簇距离.

在树状图的特定层次上进行分割, 则可得到相应的簇划分结果. 例如, 以图 9.12 中所示虚线分割树状图, 将得到包含 7 个聚类簇的结果:

$$\begin{aligned} C_1 &= \{x_1, x_{26}, x_{29}\}; & C_2 &= \{x_2, x_3, x_4, x_{21}, x_{22}\}; \\ C_3 &= \{x_{23}, x_{24}, x_{25}, x_{27}, x_{28}, x_{30}\}; & C_4 &= \{x_5, x_7\}; \\ C_5 &= \{x_9, x_{13}, x_{14}, x_{16}, x_{17}\}; & C_6 &= \{x_6, x_8, x_{10}, x_{15}, x_{18}, x_{19}, x_{20}\}; \\ C_7 &= \{x_{11}, x_{12}\}. \end{aligned}$$

回顾一下常见聚类算法

